SUPPLEMENT TO: EXPLICIT AND ASYMPTOTIC ESTIMATES FOR THE ZERO-COUNTING FUNCTION OF THE RIEMANN ZETA-FUNCTION

VÍCTOR CASAÑA-GINER

Abstract. This supplement reproduces the programs, the outputs of the 25 certificates and the input data of the certificates of the article *Explicit and asymptotic estimates for the zero-counting function of the Riemann zeta-function*. Section and equation numbers without the prefix S refer to the article. The same files are available at `https://github.com/vcasana1/PaperMath/tree/explicit-S-turing/numerics/explicit_S_turing` and at Zenodo, doi:10.5281/zenodo.23858827.

Contents

S1. Programs 2
S1.1. `certutil.py`: shared helpers, directed decimal rounding of bounds 2
S1.2. `certify_hybrid.py`: certification of the kernel inequality (with or without discs) and of C_{H} 3
S1.3. `profile_published.py`: the nodes of the profile $\mathfrak{e}_{\mathrm{P}}$ 5
S1.4. `cz_gen_pp.py`: certification of C_ζ (per-prime phase maximum) 5
S1.5. `elem_gen.py`: certification of $\sup F_u$, $\sup F_\ell$ (modes N and S) 7
S1.6. `certify_line_pp.py`: driver for one explicit line 8
S1.7. Search programs 9
S1.8. `lpcommon.py: helpers of the search programs` 9
S1.9. `explore_discs.py: the hybrid linear programme` 10
S1.10. `kernel_only.py: kernel-only optima (Remark on the infimum)` 11
S1.11. `opt_line.py: the second-order cone programme of one explicit line` 12
S2. Outputs of the certification programs 13
S2.1. Hybrid: `certify_hybrid.py hyb_m10_P.json 40` 14
S2.2. Line ℓ_5: kernel inequality 14
S2.3. Line ℓ_5: C_ζ 14
S2.4. Line ℓ_5: elementary part, mode N 14
S2.5. Line ℓ_5: elementary part, mode S 14
S2.6. Line ℓ_{20}: kernel inequality 14
S2.7. Line ℓ_{20}: C_ζ 15
S2.8. Line ℓ_{20}: elementary part, mode N 15
S2.9. Line ℓ_{20}: elementary part, mode S 15
S2.10. Line ℓ_{60}: kernel inequality 15
S2.11. Line ℓ_{60}: C_ζ 15
S2.12. Line ℓ_{60}: elementary part, mode N 15
S2.13. Line ℓ_{60}: elementary part, mode S 16
S2.14. Line ℓ_{150}: kernel inequality 16
S2.15. Line ℓ_{150}: C_ζ 16
S2.16. Line ℓ_{150}: elementary part, mode N 16
S2.17. Line ℓ_{150}: elementary part, mode S 16
S2.18. Line ℓ_{400}: kernel inequality 16
S2.19. Line ℓ_{400}: C_ζ 17
S2.20. Line ℓ_{400}: elementary part, mode N 17
S2.21. Line ℓ_{400}: elementary part, mode S 17
S2.22. Line ℓ_{6000}: kernel inequality 17
S2.23. Line ℓ_{6000}: C_ζ 17

S2.24. Line ℓ_{6000}: elementary part, mode N 17
S2.25. Line ℓ_{6000}: elementary part, mode S 17
S2.26. Grid optima of `kernel_only.py` (Remark 2.5 of the article; not certified) 18
S3. Input data of the certificates 18
S3.1. `hyb_m10_P.json` (hybrid) 18
S3.2. `hyb_m10_P_discs_rational.json` (derived) 20
S3.3. `line_L5/sol.json` (line ℓ_5) 21
S3.4. `line_L20/sol.json` (line ℓ_{20}) 21
S3.5. `line_L60/sol.json` (line ℓ_{60}) 21
S3.6. `line_L150/sol.json` (line ℓ_{150}) 22
S3.7. `line_C400/sol.json` (line ℓ_{400}) 22
S3.8. `line_D6000/sol.json` (line ℓ_{6000}) 23
References 23

S1. Programs

The programs are written in Python 3 and use `python-flint` (Arb ball arithmetic [2], FLINT [1]) at 256 bits, `numpy`, `scipy`, `cvxpy` with the solver Clarabel, and `mpmath` (in `opt_line.py` only). The certification programs are those of Sections S1.1–S1.6; every bound they print or write to a file is converted to decimal with directed rounding (`up_str` in `certutil.py`), so that the printed decimal is itself a valid bound. Each certificate begins with the SHA-256 digests of its input file and of the programs that produced it, and the versions of Python and `python-flint`; the build script of this note checks that these digests are those of the files reproduced in Appendices S1 and S3. The search programs that produced the coefficients are reproduced in Section S1.7 for completeness, but no statement of this article depends on them. All files are in the repository cited in Section 7 of the article, in the folders `hybrid_asymptotic/` and `family/`; the deposited archive is flat, and the programs locate their helper modules in either layout.

S1.1. `certutil.py`: shared helpers, directed decimal rounding of bounds.

```
# Shared helpers of the certification programs (python-flint / Arb ball arithmetic at 256 bits).
#   A(q)            exact ball of a Fraction or of an integer (floats are refused)
#   box(lo, hi)     ball containing the interval [lo, hi]
#   exact(x)        the rational value of an exact (zero-radius) arb
#   up_str(x, n)    decimal string with n significant digits that is >= every point of the ball x (directed rounding)
#   down_str(x, n)  decimal string with n significant digits that is <= every point of the ball x
#   provenance(...) lines with the SHA-256 of the input and program files and the library versions, for the record
# The .str() method of arb rounds to the nearest decimal, so it must not be used to print an upper or lower bound;
# every bound printed or written to a JSON file by the certification programs goes through up_str / down_str.
import hashlib, os, sys, platform
from fractions import Fraction as Fr
from decimal import Decimal, ROUND_CEILING, ROUND_FLOOR, localcontext
import flint
from flint import arb, ctx
ctx.prec = 256

def A(q):
    # exact ball of a Fraction or an integer; floats are refused, so that no binary approximation of a decimal enters
     silently
    if isinstance(q, Fr): return arb(q.numerator)/arb(q.denominator)
    assert isinstance(q, int), 'A() takes a Fraction or an integer, got %r' % (q,)
    return arb(q)

def box(lo, hi):
    return A(lo).union(A(hi))

def exact(x):
    assert x.is_exact() and x.is_finite(), x
    m, e = x.man_exp()
    return Fr(int(m))*Fr(2)**int(e)

def _dir_str(q, n, rounding):
    with localcontext() as c:
        c.prec = n; c.rounding = rounding
        dec = Decimal(q.numerator)/Decimal(q.denominator)      # correctly rounded to n significant digits
    return format(dec, 'f')

def up_str(x, n):
    u = exact(x.upper()); s = _dir_str(u, n, ROUND_CEILING)
```

```
    assert Fr(s) >= u, (s, u)
    return s

def down_str(x, n):
    l = exact(x.lower()); s = _dir_str(l, n, ROUND_FLOOR)
    assert Fr(s) <= l, (s, l)
    return s

def sha256(path):
    h = hashlib.sha256()
    with open(path, 'rb') as fh: h.update(fh.read())
    return h.hexdigest()

def provenance(*paths):
    # one line per file (name and SHA-256), then the versions of Python and python-flint and the working precision
    lines = ['%s sha256 %s' % (os.path.basename(p), sha256(p)) for p in paths]
    lines.append('python %s, python-flint %s, precision %d bits' % (platform.python_version(), getattr(flint, '__version__',
      '?'), ctx.prec))
    return '\n'.join(lines)

if __name__ == '__main__':
    x = arb(1)/3
    print(x.str(12, radius=False), up_str(x, 12), down_str(x, 12))
    y = -arb(1)/7
    print(y.str(12, radius=False), up_str(y, 12), down_str(y, 12))
```

S1.2. certify_hybrid.py: **certification of the kernel inequality (with or without discs) and of C_{H}.**

```
# Certification (Arb ball arithmetic, python-flint, 256 bits) of the kernel inequality
#   f(b,t) <= U(b,t) + sum_k kappa_k W_k(b,t)     for 0 <= b <= 1/2, t != 0,
# with f(b,t) = 2 atan((d+b)/t) + 2 atan((d-b)/t) - atan((2d+b)/t) - atan((2d-b)/t),
#      U(b,t) = sum_iota c_iota [F_iota(d+b,t) + F_iota(d-b,t)],  F_Rj = Re (x+it)^(-j),  F_Ij = -Im (x+it)^(-j),
#      W_k(b,t) = log^+( R_k / sqrt((c_k-1/2-b)^2 + (t+v_k)^2) ),
# and of the constant C_H = c_R1/(2 pi) + (1/pi) sum_k kappa_k J_k,  J_k = (1/pi) int_0^pi e_P(c_k + R_k cos th) dth,
# where e_P is the piecewise linear profile of profile_published.py.  With an empty list of discs the program certifies
# the kernel inequality f <= U alone and prints C_1 = c_R1/(2 pi).
#
# Usage:  python3 certify_hybrid.py <solution.json> [T1]
#   solution.json: d, B (basis), c (coefficients, as decimal strings that round-trip to the optimiser's floats), discs [(c_k,
     R_k,v_k)], kappa; T1 defaults to 40.
# Method (Proposition "Certification of the kernel inequality" and Section "The hybrid" of the article):
#   (a) on [0,1/2] x (0,T1] and [0,1/2] x [-T1,0): centred form H(centre) + grad H(box) . half-widths, bisection;
#   (b) for |t| >= T1, u = 1/t: t^2 U - 4 d^3 u - K5 |u|^3 > 0 on [0,1/2] x [0,1/T1] and [0,1/2] x [-1/T1,0], bisection;
#       this needs T1 >= 2d + 1/2 (convergence of the arctan series) and W_k = 0 for |t| >= T1.
import sys, os, json, time
from fractions import Fraction as Fr
from flint import arb, acb
from certutil import A, box, up_str, down_str, exact, provenance

sol = json.load(open(sys.argv[1])); T1 = Fr(sys.argv[2]) if len(sys.argv) > 2 else Fr(40)
d = Fr(sol['d']).limit_denominator(10**6); B = [tuple(x) for x in sol['B']]
c = [Fr(x) for x in sol['c']]                                          # exact rational values of the binary floats
     recovered from the round-tripping JSON decimals
discs = [(Fr(cc).limit_denominator(10**4), Fr(R).limit_denominator(10**4), Fr(v).limit_denominator(10**4)) for cc, R, v in
     sol['discs']]
discs = [(cc, min(R, cc-Fr(1, 2)), v) for cc, R, v in discs]           # rational rounding must keep c_k - R_k >= 1/2 (
     decreasing R_k lowers W_k; J_k is recomputed below from the final rational data)
kap = [Fr(x) for x in sol['kappa']]
# integrity of the data
assert len(c) == len(B) and len(discs) == len(kap)
assert B[0] == ('R', 1) and ('R', 2) in B and all(k in ('R', 'I') and j >= 1 for k, j in B) and ('I', 1) not in B
assert d > Fr(1, 2)
assert all(k >= 0 for k in kap) and all(R > 0 and cc-R >= Fr(1, 2) and cc > 1 for cc, R, v in discs)
assert T1 >= 2*d+Fr(1, 2)
assert all(abs(v)+R <= T1 for cc, R, v in discs)                        # then W_k = 0 for |t| >= T1
JMAX = max(j for _, j in B)
dA = A(d); half = A(Fr(1, 2)); PI = arb.pi()
cA = [A(x) for x in c]; kA = [A(x) for x in kap]; dsA = [(A(cc), A(R).log(), A(v)) for cc, R, v in discs]

def powers(x, t, n):
    # [(x+it)^0, (x+it)^-1, ..., (x+it)^-n]
    w = acb(1)/acb(x, t); pw = [acb(1)]
    for _ in range(n): pw.append(pw[-1]*w)
    return pw
def kernel(b, t):
    U = arb(0)
    for x in (dA+b, dA-b):
        pw = powers(x, t, JMAX)
        for i, (k, j) in enumerate(B):
            U += cA[i]*(pw[j].real if k == 'R' else -pw[j].imag)
    return U
```

```
def fbox(b, t, sgn):
    # f on the closure of t > 0 (sgn = +1) or of t < 0 (sgn = -1), in the form regular at t = 0
    return sgn*PI-2*(t/(dA+b)).atan()-2*(t/(dA-b)).atan()+(t/(2*dA+b)).atan()+(t/(2*dA-b)).atan()
def Wlow(b, t):
    # lower bound of sum_k kappa_k W_k on the box (W_k >= 0, so dropping a term is safe)
    s = arb(0)
    for (cc, lR, v), k in zip(dsA, kA):
        val = lR-((cc-half-b)**2+(t+v)**2).log()/2
        lo = val.lower()
        if lo > 0: s += k*lo
    return s
def grad(b, t):
    # partial derivatives of U - f with respect to b and t, on a box
    gb = arb(0); gt = arb(0)
    for x, sg in ((dA+b, 1), (dA-b, -1)):
        pw = powers(x, t, JMAX+1)
        for i, (k, j) in enumerate(B):
            dx = -j*pw[j+1]; dt = acb(0, -j)*pw[j+1]                      # d/dx (x+it)^-j,  d/dt (x+it)^-j
            if k == 'R': gb += cA[i]*sg*dx.real; gt += cA[i]*dt.real
            else: gb -= cA[i]*sg*dx.imag; gt -= cA[i]*dt.imag
    y1 = dA+b; y2 = dA-b; y3 = 2*dA+b; y4 = 2*dA-b; t2 = t*t
    ft = -2*y1/(y1*y1+t2)-2*y2/(y2*y2+t2)+y3/(y3*y3+t2)+y4/(y4*y4+t2)
    fb = 2*t/(y1*y1+t2)-2*t/(y2*y2+t2)-t/(y3*y3+t2)+t/(y4*y4+t2)
    return gb-fb, gt-ft
def Hcentered(b0, b1, t0, t1, sgn):
    bm = (b0+b1)/2; tm = (t0+t1)/2; b = box(b0, b1); t = box(t0, t1)
    H0 = kernel(A(bm), A(tm))-fbox(A(bm), A(tm), sgn)
    gb, gt = grad(b, t)
    H = H0+gb*box(-(b1-b0)/2, (b1-b0)/2)+gt*box(-(t1-t0)/2, (t1-t0)/2)
    return H+Wlow(b, t)
def cert_main(blo, bhi, tlo, thi, sgn, maxdepth=60):
    stack = [(blo, bhi, tlo, thi, 0)]; n = 0
    while stack:
        b0, b1, t0, t1, dep = stack.pop(); n += 1
        H = Hcentered(b0, b1, t0, t1, sgn)
        if H > 0: continue
        if dep >= maxdepth: raise RuntimeError('fail at b=[%s,%s] t=[%s,%s] H=%s' % (float(b0), float(b1), float(t0), float(
     t1), H))
        wb = (b1-b0)/Fr(1, 20); tm = max(abs(t0), abs(t1)); wt = (t1-t0)/max(Fr(1, 20), tm*Fr(3, 10))
        if wb >= wt: m_ = (b0+b1)/2; stack += [(b0, m_, t0, t1, dep+1), (m_, b1, t0, t1, dep+1)]
        else: m_ = (t0+t1)/2; stack += [(b0, b1, t0, m_, dep+1), (b0, b1, m_, t1, dep+1)]
    return n

# tail |t| >= T1, u = 1/t:  t^2 (U - f) >= t^2 U - 4 d^3 u - K5 |u|^3,  with t^2 U = sum_iota c_iota [Phi_iota(d+b,u) +
     Phi_iota(d-b,u)],
# Phi_R1 = x/(1+x^2 u^2),  Phi_iota = Re / -Im of u^(j-2) (xu+i)^(-j) for j >= 2;  K5 = (1/5) sum_i |alpha_i| y_i^5 (Lemma "
     The tail of the kernel").
def tailval(b, u):
    V = arb(0)
    for x in (dA+b, dA-b):
        for i, (k, j) in enumerate(B):
            if j == 1: V += cA[i]*x/(1+x*x*u*u)
            else:
                z = acb(u)**(j-2)*(acb(1)/acb(x*u, 1))**j
                V += cA[i]*(z.real if k == 'R' else -z.imag)
    return V
K5 = (2*(dA+half)**5+2*dA**5+(2*dA+half)**5+(2*dA)**5)/5
def cert_tail(u0, u1, maxdepth=40):
    assert max(abs(u0), abs(u1))*(2*d+Fr(1, 2)) <= 1                    # |y u| <= 1 for every y in {d+b, d-b, 2d+b, 2d-b}
    stack = [(Fr(0), Fr(1, 2), u0, u1, 0)]; n = 0
    while stack:
        b0, b1, a0, a1, dep = stack.pop(); n += 1
        b = box(b0, b1); u = box(a0, a1)
        H = tailval(b, u)-4*dA**3*u-K5*abs(u)**3
        if H > 0: continue
        if dep >= maxdepth: raise RuntimeError('tail fail b=[%s,%s] u=[%s,%s] H=%s' % (float(b0), float(b1), float(a0), float
     (a1), H))
        if (b1-b0)*8 >= (a1-a0)*T1: m_ = (b0+b1)/2; stack += [(b0, m_, a0, a1, dep+1), (m_, b1, a0, a1, dep+1)]
        else: m_ = (a0+a1)/2; stack += [(b0, b1, a0, m_, dep+1), (b0, b1, m_, a1, dep+1)]
    return n

def Jexact(nodes, cc, R):
    # (1/pi) int_0^pi e(cc + R cos th) dth for e piecewise linear through nodes, e = 0 for sigma >= 1; needs cc - R >= 1/2
    assert nodes[0][0] == Fr(1, 2) and nodes[-1] == (Fr(1), Fr(0)) and all(p[0] < q[0] for p, q in zip(nodes, nodes[1:]))
    tot = arb(0); ccA = A(cc); RA = A(R)
    lo, hi = cc-R, min(cc+R, Fr(1))
    for (p, ep), (q, eq) in zip(nodes, nodes[1:]):
        a, bq = max(p, lo), min(q, hi)
        if a >= bq: continue
        Bs = (eq-ep)/(q-p); As = ep-Bs*p                                  # e = As + Bs sigma on [p, q]
        tha = A((a-cc)/R).acos(); thb = A((bq-cc)/R).acos()               # tha >= thb
```

```
        tot += (A(As)+A(Bs)*ccA)*(tha-thb)+A(Bs)*RA*(tha.sin()-thb.sin())
    return tot/PI

if __name__ == '__main__':
    t_ = time.time()
    print(provenance(sys.argv[1], __file__, os.path.join(os.path.dirname(os.path.abspath(__file__)), 'certutil.py'), *([os.
     path.join(os.path.dirname(os.path.abspath(__file__)), 'profile_published.py')] if sol['discs'] else []))
    if discs:
        assert sol['prof'] == 'P', 'only the published-source profile e_P is supported'
        from profile_published import nodes_published
        nodes = nodes_published()
        J = [Jexact(nodes, cc, R) for cc, R, v in discs]
    else: J = []
    C = cA[0]/(2*PI)+sum((k*j for k, j in zip(kA, J)), arb(0))/PI
    print('d =', d, ' m =', JMAX, ' discs used:', len(discs), (' profile e_P (profile_published.py)' if discs else ''))
    print('C = c_R1/(2pi) + sum kappa_k J_k/pi =', C.str(25), ' (ball, midpoint and radius)')
    print('   radius of the ball:', C.rad().str(3), '  enclosure of C with directed rounding: [%s, %s]' % (down_str(C, 20),
     up_str(C, 20)))
    print('kernel part c_R1/(2pi) =', (cA[0]/(2*PI)).str(20))
    lim = 2*dA*cA[0]-2*cA[B.index(('R', 2))]; print('limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 =', lim.str(10), '
     positive:', lim > 0)
    n1 = cert_main(Fr(0), Fr(1, 2), Fr(0), T1, +1); print('t in (0,T1]: certified, boxes', n1, flush=True)
    n2 = cert_main(Fr(0), Fr(1, 2), -T1, Fr(0), -1); print('t in [-T1,0): certified, boxes', n2, flush=True)
    n3 = cert_tail(Fr(0), 1/T1); n4 = cert_tail(-1/T1, Fr(0)); print('|t|>=T1: certified, boxes', n3+n4)
    Cup = up_str(C, 12); assert exact(C.upper()) <= Fr(Cup)
    print('all certified;  C <', Cup, '  time %.0fs' % (time.time()-t_))
```

S1.3. `profile_published.py`: **the nodes of the profile** $\mathfrak{e}_{\mathrm{P}}$.

```
# Upper bound for mu(sigma) on [1/2,1] from published sources only, as a piecewise linear function through nodes
# (sigma_i, e_i) with mu(sigma_i) <= e_i a theorem in each case:
#  - 1/2: 13/84 (Bourgain 2017, Thm 5);
#  - breakpoints of Trudgian-Yang, Toward optimal exponent pairs, arXiv:2306.05599v3, Thm 2.4 (pieces are continuous);
#  - l-k for the exponent pairs (k,l) of Tao-Trudgian-Yang, arXiv:2501.16779v1, Thm 20, via Lemma 23 (mu(l-k) <= k);
#  - Heath-Brown 2017 (Proc. Steklov Inst. Math. 296), Thm 2: pairs (2/((n-1)^2(n+2)), 1-(3n-2)/(n(n-1)(n+2))+eps), n >= 3,
#    giving mu(sigma_n) <= mu_n, sigma_n = 1-(3n^2-3n+2)/(n(n-1)^2(n+2)), mu_n = 2/((n-1)^2(n+2)) (continuity of mu), 6 <= n
     <= 399;
#  - (1, 0) (zeta(1+it) << log t).
# The profile is the lower convex hull of these points (nodes above a chord of two others are dropped: the chord is a
# valid bound by the Phragmen-Lindelof interpolation lemma, and smaller).
from fractions import Fraction as Fr
TY = [(Fr(1,2),Fr(88225,153852),Fr(31,84),Fr(3,7)),(Fr(88225,153852),Fr(521,796),Fr(220633,620612),Fr(62831,155153)),
 (Fr(521,796),Fr(53141,76066),Fr(1333,3825),Fr(1508,3825)),(Fr(53141,76066),Fr(3620,5119),Fr(405,1202),Fr(227,601)),
 (Fr(3620,5119),Fr(52209,69128),Fr(49318855,170145110),Fr(52938216,170145110)),(Fr(52209,69128),Fr(1389,1736),Fr
     (471957,1682490),Fr(502648,1682490)),
 (Fr(1389,1736),Fr(134765,163248),Fr(2841,10316),Fr(3016,10316)),(Fr(134765,163248),Fr(18193,21906),Fr(859,3214),Fr(908,3214)
     ),
 (Fr(18193,21906),Fr(249,280),Fr(5*8707,180277),Fr(5*9067,180277)),(Fr(249,280),Fr(277,300),Fr(29,130),Fr(30,130))]
PAIRS = [(Fr(89,1282),Fr(997,1282)),(Fr(652397,9713986),Fr(7599781,9713986)),(Fr(10769,351096),Fr(609317,702192)),(Fr
     (89,3478),Fr(15327,17390))]
def hb(n): return 1-Fr(3*n*n-3*n+2, n*(n-1)**2*(n+2)), Fr(2, (n-1)**2*(n+2))
def raw_nodes(nmax=400):
    for (l1,h1,a1,b1),(l2,h2,a2,b2) in zip(TY,TY[1:]): assert a1-b1*h1 == a2-b2*h1
    nodes = {}
    for lo,hi,a,b in TY: nodes[lo] = a-b*lo; nodes[hi] = a-b*hi
    for k,l in PAIRS: nodes[l-k] = min(nodes.get(l-k, Fr(1)), k)
    for n in range(6, nmax): s,m = hb(n); nodes[s] = min(nodes.get(s, Fr(1)), m)
    nodes[Fr(1)] = Fr(0)
    return sorted(nodes.items())
def nodes_published(nmax=400):
    P = raw_nodes(nmax); hull = []
    def cross(o,a,b): return (a[0]-o[0])*(b[1]-o[1])-(a[1]-o[1])*(b[0]-o[0])
    for p in P:
        while len(hull) >= 2 and cross(hull[-2], hull[-1], p) <= 0: hull.pop()
        hull.append(p)
    return hull
if __name__ == '__main__':
    H = nodes_published()
    print(len(H), 'nodes; first ones:')
    for s,e in H[:14]: print('  sigma = %-20s  e = %-18s  (%.6f, %.6f)' % (s, e, s, e))
```

S1.4. `cz_gen_pp.py`: **certification of** C_ζ **(per-prime phase maximum).**

```
# Rigorous upper bound (Arb ball arithmetic, 256 bits) for the Euler-product constant of a kernel,
#   C_zeta = sum_{p <= P0} M_p + sum_{n = p^k, p > P0} Lambda(n) n^(-sigma1) h(log n) + tail,
# with h(L) = sqrt(a(L)^2 + b(L)^2),  a(L) = -(1/pi) sum_{(R,j)} c_{R,j} L^(j-1)/(j-1)!,
# b(L) = (1/pi) [ (e^(-dL) - 2)/L - sum_{(I,j)} c_{I,j} L^(j-1)/(j-1)! ],  and M_p the maximum over the phase x of
#   f_p(x) = sum_{k: p^k <= N0} (log p) p^(-k sigma1) [ a(k log p) cos(kx) + b(k log p) sin(kx) ]
# (per-prime phase maximum, Lemma "The contribution of one prime"), bounded by the maximum on a grid of M points plus a
# Lipschitz correction.  The sum over the prime powers n <= N0 is exact; the tail n > N0 uses Lemma "The tail" with the
# non-increasing step majorant G of g(x) = h(log x) x^(-sigma1) on [N0, e^LB], the certified monotonicity of g beyond e^LB
```

```
# (Lemma "Monotonicity of g") and psi(x) < 1.03883 x (Rosser-Schoenfeld 1962, Thm 12).
#
# Usage:  python3 cz_gen_pp.py <solution.json> <N0> [P0]      (the article uses N0 = 1e8, P0 = 1000)
# Output: <solution>_czpp.json with the upper bound 'Czeta_upper' (decimal rounded upwards).
import sys, os, json, time, numpy as np
from fractions import Fraction as Fr
from math import factorial
from flint import arb, acb
sys.path.append(os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'hybrid_asymptotic'))   # certutil.py lives
     there (or next to this file in the flat archive)
from certutil import A, up_str, provenance

sol = json.load(open(sys.argv[1])); N0 = int(float(sys.argv[2])); P0 = int(sys.argv[3]) if len(sys.argv) > 3 else 1000
print(provenance(sys.argv[1], __file__, os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'hybrid_asymptotic', '
     certutil.py') if not os.path.exists(os.path.join(os.path.dirname(os.path.abspath(__file__)), 'certutil.py')) else os.
     path.join(os.path.dirname(os.path.abspath(__file__)), 'certutil.py')), flush=True)
d = Fr(sol['d']).limit_denominator(10**6); B = [tuple(b) for b in sol['B']]; c = [A(Fr(x)) for x in sol['c']]
assert len(c) == len(B) and B[0] == ('R', 1) and ('I', 1) not in B and d > Fr(1, 2) and P0 <= N0
PI = arb.pi(); dA = A(d); s1 = A(Fr(1, 2)+d); alpha = s1-1                # alpha = sigma1 - 1 > 0
# polynomial coefficients (in L) of pi*a and of the polynomial part of pi*b
DEG = max(j for _, j in B)-1
pa = [arb(0)]*(DEG+1); pb = [arb(0)]*(DEG+1)
for (k, j), cc in zip(B, c):
    if k == 'R': pa[j-1] -= cc/factorial(j-1)
    else: pb[j-1] -= cc/factorial(j-1)
def poly(p, L): return sum((p[i]*L**i for i in range(len(p))), arb(0))
def h(L):
    a0 = poly(pa, L)/PI; b0 = (((-dA*L).exp()-2)/L+poly(pb, L))/PI
    return (a0*a0+b0*b0).sqrt()

t0 = time.time()
# primes up to N0 (sieve of Eratosthenes)
s = np.ones(N0+1, np.bool_); s[:2] = False
for i in range(2, int(N0**0.5)+1):
    if s[i]: s[i*i::i] = False
P = np.nonzero(s)[0]; del s
# exact sum over the prime powers n <= N0, with the per-prime phase maximum for p <= P0
M = 4096
xs = [2*PI*i/M for i in range(M)]; hx = PI/M                              # grid points and half-spacing
tot = arb(0); psi = arb(0); gain = arb(0)
for p in P.tolist():
    lp = arb(p).log(); n = p; k = 1; coef = []; hyp = arb(0)
    while n <= N0:
        L = k*lp; w = lp*(-s1*L).exp(); a0 = poly(pa, L)/PI; b0 = (((-dA*L).exp()-2)/L+poly(pb, L))/PI
        coef.append((k, w*a0, w*b0)); hyp += w*(a0*a0+b0*b0).sqrt(); psi += lp; n *= p; k += 1
    if p <= P0:
        # max_x f_p(x) <= max_i f_p(x_i) + hx * sum_k k |(a_k, b_k)|   (Lipschitz constant of f_p)
        Lip = sum((kk*(a*a+b*b).sqrt() for kk, a, b in coef), arb(0))
        best = None
        for x in xs:
            v = sum((a*(kk*x).cos()+b*(kk*x).sin() for kk, a, b in coef), arb(0))
            best = v.upper() if best is None or v.upper() > best else best
        mb = best+hx*Lip
        mb = mb if mb < hyp else hyp                                      # the trivial bound sum_k |(a_k,b_k)| is always
     valid
        gain += hyp-mb; tot += mb
    else:
        tot += hyp
print('per-prime phase maximum for p <=', P0, ': gain', gain.str(8), flush=True)
print('primes', len(P), ' partial sum (n<=%d) =' % N0, tot.str(15), ' psi(N0) =', psi.str(15), ' %.0fs' % (time.time()-t0),
     flush=True)

# monotonicity of g on [e^LB, oo) (Lemma "Monotonicity of g"): in u = 1/L in [0, 1/LB], num(u)/low(u) < sigma1 with
#   num(u) = u sum_{i>=1} i q_i u^(D-i) + 2 u^(D+2) + d u^(D+1),   low(u) = |p_D| - sum_{i<D} |p_i| u^(D-i) - eps_P u^(D+1),
# where P is the one of pi*a, pi*b of larger degree D (pi*a if equal), q_i = |a_i| + |b_i|, eps_P = 2 if P = pi*b else 0.
L0 = arb(N0).log()
DA = max([i for i in range(DEG+1) if pa[i] != 0], default=-1); DB = max([i for i in range(DEG+1) if pb[i] != 0], default=-1)
q = [abs(pa[i])+abs(pb[i]) for i in range(DEG+1)]
if DA >= DB: D, pp, extra = DA, pa, 0
else: D, pp, extra = DB, pb, 2
assert D == DEG, 'the top coefficient of the kernel vanishes'
LB = None
for LBc in [2000.0, 5000.0, 2e4, 1e5, 1e6, 1e7]:
    u = arb(0).union(1/arb(LBc))
    num = u*sum((i*q[i]*u**(D-i) for i in range(1, DEG+1)), arb(0))+2*u**(D+2)+dA*u**(D+1)
    low = abs(pp[D])-sum((abs(pp[i])*u**(D-i) for i in range(D)), arb(0))-extra*u**(D+1)
    if low > 0 and num/low < s1: LB = LBc; ratio = num/low; break
assert LB is not None, 'monotonicity of g could not be certified'
print('g monotone on [%g, oo): crude bound %s < sigma1' % (LB, ratio.str(6)))

# step majorant G >= g on [N0, e^LB]: on [e^u_i, e^u_{i+1}] the running maximum, from the right, of upper bounds of g on
```

```
# the small boxes, started at g(e^LB) (g is non-increasing beyond e^LB), so that G is non-increasing on [N0, oo)
def cpoly(p, Lb):   # centred form of a polynomial on a ball: p(m) + p'(Lb)(Lb - m)
    m = arb(Lb.mid()); dp = [p[i]*i for i in range(1, len(p))]
    return poly(p, m)+poly(dp, Lb)*(Lb-m)
def h_up(Lb):
    a0 = cpoly(pa, Lb)/PI; b0 = (((-dA*Lb).exp()-2)/Lb+cpoly(pb, Lb))/PI
    au = abs(a0).upper(); bu = abs(b0).upper()
    return (au*au+bu*bu).sqrt().upper()
edges = [float(L0.lower())-1e-9]
while edges[-1] < LB: edges.append(min(edges[-1]+max(0.003, 0.0002*edges[-1]), LB))
gup = []
for i in range(len(edges)-1):
    Lb = arb(edges[i]).union(arb(edges[i+1])); gup.append(h_up(Lb)*(-s1*arb(edges[i])).exp())
run = h(arb(LB))*(-s1*arb(LB)).exp(); G = [None]*len(gup)
for i in range(len(gup)-1, -1, -1):
    run = run if run > gup[i] else (gup[i] if gup[i] > run else run.union(gup[i]).upper()); G[i] = run.upper()
Istep = sum((G[i]*(arb(edges[i+1]).exp()-arb(edges[i]).exp()) for i in range(len(G))), arb(0))   # int_{N0}^{e^LB} G dx

# int_{e^LB}^oo g dx  <=  int_{e^LB}^{e^L1} g dx + int_{e^L1}^oo gtilde dx   with L1 = 800 and the majorant
#   gtilde = (1/pi) (sum_i q_i L^i + 3) e^(-sigma1 L) >= g on [e^L1, oo)   (from h <= |a| + |b| and 2/L <= 3);
# when LB > L1 the first integral is negative, and the inequality holds because gtilde >= g on [e^L1, e^LB] as well.
L1 = arb(800)
assert LB >= 800
def fz(z, analytic):
    a0 = sum((acb(pa[i])*z**i for i in range(len(pa))), acb(0))/PI
    b0 = (((-acb(dA)*z).exp()-2)/z+sum((acb(pb[i])*z**i for i in range(len(pb))), acb(0)))/PI
    return (a0*a0+b0*b0).sqrt(analytic=analytic)*((1-acb(s1))*z).exp()
Icancel = acb.integral(fz, acb(L1), acb(LB))                                  # int_{L1}^{LB} h(L) e^{(1-sigma1) L} dL = int_{e^
    L1}^{e^LB} g dx
assert Icancel.is_finite() and Icancel.imag.contains(0)
def inc(k, X): return (-alpha*X).exp()*sum((arb(factorial(k))/factorial(i)*X**i/alpha**(k-i+1) for i in range(k+1)), arb(0))
      # int_X^oo L^k e^{-alpha L} dL
Ifar = (sum((q[i]*inc(i, L1) for i in range(DEG+1)), arb(0))+3*inc(0, L1))/PI   # int_{e^L1}^oo gtilde dx
Itail = Istep-Icancel.real+Ifar                                               # >= int_{N0}^oo G dx
# Lemma "The tail": sum_{n > N0} Lambda(n) g(n) <= G(N0) (1.03883 N0 - psi(N0)) + 1.03883 int_{N0}^oo G dx
tail = G[0]*(arb('1.03883')*N0-psi)+arb('1.03883')*Itail
Cz = tot+tail
print('tail <=', tail.str(10), ' (int_{N0}^{e^LB} G =', Istep.str(10), ', int_{e^L1}^{e^LB} g =', Icancel.real.str(10), ',
    far tail <=', Ifar.str(5), ')')
print('C_zeta <', up_str(Cz, 12), '   time %.0fs' % (time.time()-t0))
json.dump({'Czeta_upper': up_str(Cz, 15), 'N0': N0, 'P0': P0}, open(sys.argv[1].replace('.json', '_czpp.json'), 'w'))
```

S1.5. **elem_gen.py: certification of $\sup F_u$, $\sup F_\ell$ (modes N and S).**

```
# Rigorous upper bound (Arb ball arithmetic, 256 bits) for sup_{T >= 1} F_u(T) and sup_{T >= 1} F_l(T), where
#   F_u(T) = E_u(T) - A1 log T,   F_l(T) = -E_l(T) - A1 log T,
#   E_u(T) = (1/pi)(2 Ael(s1) - Ael(s2)) - main(T) + (1/pi) sum_iota c_iota Phi_iota(D_j(s1)),
#   E_l(T) = (1/pi)(2 Ael(s1) - Ael(s2)) - main(T) - (1/pi) [ sum_{(R,j)} c Re D_j(s1) - sum_{(I,j)} c (-Im D_j(s1)) ],
#   Ael(s) = arg s + arg(s-1) - (T/2) log pi + Im log Gamma(s/2),   main(T) = (T/2pi) log(T/(2 pi e)),
#   D_1(s) = 1/s + 1/(s-1) - (1/2) log pi + (1/2) psi(s/2),   D_j(s) = s^-j + (s-1)^-j + ((-1)^(j-1)/(j-1)!) 2^-j psi^(j-1)(s
    /2) (j >= 2),
#   s1 = 1/2 + d + iT,  s2 = 1/2 + 2d + iT,  Phi_R = Re,  Phi_I = -Im.
# Mode S replaces main(T) by theta(T)/pi + 1 (then the bounds are for |S(T)|).
# [1, T0]: centred form F(mid) + F'(box) * [-half-width, half-width] on geometric intervals of ratio 1001/1000;
# T >= T0: Lemma "The elementary part for large T": F_u <= 7/8 - C1 log(2 pi) + K/T, F_l <= -7/8 - C1 log(2 pi) + K/T
# (mode S: -C1 log(2 pi) + (K + (1/16 + 1/8 + 1/3)/pi)/T), with the constant K of that lemma; needs A1 >= C1 = c_R1/(2 pi).
#
# Usage:  python3 elem_gen.py <solution.json> <A1> [T0] [N|S]      (the article uses T0 = 10000)
# Output: <solution>_elem.json or <solution>_elemS.json with 'Fsup_upper' (decimal rounded upwards).
import sys, os, json, time
from fractions import Fraction as Fr
from math import factorial
from flint import arb, acb
sys.path.append(os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'hybrid_asymptotic'))   # certutil.py lives
    there (or next to this file in the flat archive)
from certutil import A, up_str, provenance

sol = json.load(open(sys.argv[1])); A1 = A(Fr(sys.argv[2])); T0 = int(sys.argv[3]) if len(sys.argv) > 3 else 10**4
MODE = sys.argv[4] if len(sys.argv) > 4 else 'N'
print(provenance(sys.argv[1], __file__, os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'hybrid_asymptotic', '
    certutil.py') if not os.path.exists(os.path.join(os.path.dirname(os.path.abspath(__file__)), 'certutil.py')) else os.
    path.join(os.path.dirname(os.path.abspath(__file__)), 'certutil.py')), flush=True)
assert MODE in ('N', 'S') and T0 >= 1
d = Fr(sol['d']).limit_denominator(10**6); B = [tuple(b) for b in sol['B']]; c = [A(Fr(x)) for x in sol['c']]
assert len(c) == len(B) and B[0] == ('R', 1) and ('I', 1) not in B and d > Fr(1, 2)
J = max(j for _, j in B)
PI = arb.pi(); s1r = A(Fr(1, 2)+d); s2r = A(Fr(1, 2)+2*d); LOGPI = PI.log()
cR1 = c[0]; C1 = cR1/(2*PI)
assert A1 >= C1, 'A1 must be >= c_R1/(2 pi)'

def psi(k, z): return z.polygamma(k)
```

```
def Dj(j, s, ps):   # ps[k] = psi^(k)(s/2)
    if j == 1: return 1/s+1/(s-1)-LOGPI/2+ps[0]/2
    return s**(-j)+(s-1)**(-j)+arb((-1)**(j-1))/factorial(j-1)/2**j*ps[j-1]
def dDj(j, s, ps):  # d/ds D_j(s)
    if j == 1: return -s**-2-(s-1)**-2+ps[1]/4
    return -j*s**(-j-1)-j*(s-1)**(-j-1)+arb((-1)**(j-1))/factorial(j-1)/2**(j+1)*ps[j]
def combine(vals):  # sum_{(R,j)} c Re(.) and sum_{(I,j)} c (-Im(.))
    R = sum((cc*vals[i].real for i, ((k, j), cc) in enumerate(zip(B, c)) if k == 'R'), arb(0))
    I = sum((cc*(-vals[i].imag) for i, ((k, j), cc) in enumerate(zip(B, c)) if k == 'I'), arb(0))
    return R, I
def F_at(T):
    # (F_u(T), F_l(T)) at an exact T
    s1 = acb(s1r, T); s2 = acb(s2r, T)
    Ael = lambda s: (s.log().imag+(s-1).log().imag)-T/2*LOGPI+(s/2).lgamma().imag
    base = (2*Ael(s1)-Ael(s2))/PI-T/(2*PI)*(T/(2*PI*arb(1).exp())).log()
    if MODE == 'S':   # replace main(T) by theta(T)/pi + 1
        theta = acb(arb(1)/4, T/2).lgamma().imag-T/2*LOGPI
        base = base+T/(2*PI)*(T/(2*PI*arb(1).exp())).log()-theta/PI-1
    ps = [psi(k, s1/2) for k in range(J)]
    R, I = combine([Dj(j, s1, ps) for (_, j) in B])
    return base+(R+I)/PI-A1*T.log(), -(base-(R-I)/PI)-A1*T.log()
def dF_ball(Tb):
    # (F_u'(T), F_l'(T)) on a ball Tb of heights; the polygammas on the ball by the mean value form psi^(k)(z_m) + psi^(k+1)(
     z_ball) (z - z_m)
    Tm = arb(Tb.mid()); rad = Tb-Tm
    def P(k, sr):
        zm = acb(sr, Tm)/2; zb = acb(sr, Tb)/2
        return psi(k, zm)+psi(k+1, zb)*acb(0, rad/2)
    s1 = acb(s1r, Tb); s2 = acb(s2r, Tb)
    dAel = lambda s, sr: ((1/s).real+(1/(s-1)).real)-LOGPI/2+P(0, sr).real/2      # d/dT Ael(sr + iT)
    dbase = (2*dAel(s1, s1r)-dAel(s2, s2r))/PI-(Tb/(2*PI)).log()/(2*PI)
    if MODE == 'S':   # d/dT [main - theta/pi] = (1/2pi) log(T/2pi) - (1/2pi) Re psi(1/4+iT/2) + (1/2pi) log pi
        zm = acb(arb(1)/4, Tm/2); zb = acb(arb(1)/4, Tb/2)
        P0 = psi(0, zm)+psi(1, zb)*acb(0, rad/2)
        dbase = dbase+(Tb/(2*PI)).log()/(2*PI)-(P0.real-LOGPI)/(2*PI)
    ps = [P(k, s1r) for k in range(J+1)]
    R, I = combine([acb(0, 1)*dDj(j, s1, ps) for (_, j) in B])              # d/dT = i d/ds
    return dbase+(R+I)/PI-A1/Tb, -(dbase-(R-I)/PI)-A1/Tb

# [1, T0]
t0 = time.time(); ratio = Fr(1001, 1000); a = Fr(1); n = 0; supu = None; supl = None
while a < T0:
    b = min(a*ratio, Fr(T0)); m = (a+b)/2; dl = (b-a)/2
    fu, fl = F_at(A(m)); gu, gl = dF_ball(A(a).union(A(b)))
    eu = (fu+gu*A(dl)*arb(0, 1)).upper(); el = (fl+gl*A(dl)*arb(0, 1)).upper()
    supu = eu if supu is None or eu > supu else supu; supl = el if supl is None or el > supl else supl
    a = b; n += 1
print('intervals', n, ' sup F_u on [1,T0] <=', up_str(supu, 10), '  sup F_l <=', up_str(supl, 10))
# T >= T0 (Lemma "The elementary part for large T")
T = A(Fr(T0)); s1 = s1r; s2 = s2r
K = (2*s2-1)/PI+(s1*s1/2+(s2-1)*s2/2+1)/PI+(2*s2*s2/4+(s1-1)*s1)/PI        # arguments and Gamma factors
K += abs(cR1)/PI*((2*s1-1)+s1*s1/4+arb(8)/3)/T                             # j = 1: |delta_1| <= B/T^2 <= B/(T0 T)
for (k, j), cc in zip(B, c):
    if j == 1: continue
    # |D_j(s1)| <= 2/T^j + |psi^(j-1)(z1)| / ((j-1)! 2^j),  |psi^(k)(z1)| <= (k-1)! (2/T)^k + k! (2/T)^(k+1)/2 + (k+1)! (4/(3
     T)) (4/T)^(k+1);
    # T times the bound is non-increasing in T, so the bound at T0 times T0/T is valid for T >= T0
    kk = j-1
    ps = arb(factorial(kk-1))*(2/T)**kk+arb(factorial(kk))*(2/T)**(kk+1)/2+arb(factorial(kk+1))*(4/(3*T))*(4/T)**(kk+1)
    Dbound = 2/T**j+ps/(factorial(j-1)*2**j)
    K += abs(cc)/PI*Dbound*T
Finf_u = arb(7)/8-C1*(2*PI).log(); Finf_l = -arb(7)/8-C1*(2*PI).log()
if MODE == 'S':   # theta/pi + 1 - main = 7/8 + varpi/pi, |varpi(T)| <= (1/16 + 1/8 + 1/3)/T (Lemma "The theta function")
    Finf_u = -C1*(2*PI).log(); Finf_l = -C1*(2*PI).log(); K = K+(arb(1)/16+arb(1)/8+arb(1)/3)/PI
tu = (Finf_u+K/T).upper(); tl = (Finf_l+K/T).upper()
S = max(supu, supl, tu, tl)
print('T>=T0: F_u <=', up_str(tu, 10), ' F_l <=', up_str(tl, 10), ' (K =', K.str(6), ')')
print('SUP over T>=1 of max(F_u,F_l) <=', up_str(S, 12), '   time %.0fs' % (time.time()-t0))
json.dump({'A1': sys.argv[2], 'T0': T0, 'mode': MODE, 'Fsup_upper': up_str(S, 15)}, open(sys.argv[1].replace('.json', '_elem%
     s.json' % ('' if MODE == 'N' else 'S')), 'w'))
```

S1.6. **certify_line_pp.py: driver for one explicit line.**

```
# Driver: certify one explicit line |N(T) - main(T)| <= A1 log T + A3_N and |S(T)| <= A1 log T + A3_S for all T >= 1.
#   1. certify_hybrid.py (no discs): the kernel inequality f <= U, with T1 = 100 (T1 = 2000 if that run fails);
#   2. cz_gen_pp.py: C_zeta < Czeta_upper  (N0 = 1e8, per-prime phase maximum for p <= 1000);
#   3. elem_gen.py, modes N and S: sup_{T>=1} max(F_u, F_l) < Fsup_upper  (T0 = 1e4);
#   4. A1 = C1 rounded upwards to 5 decimals, A3 = Czeta_upper + Fsup_upper rounded upwards to 3 decimals (exact decimal
     arithmetic).
# Usage:  python3 certify_line_pp.py <tag> <solution.json>                  (writes line_<tag>/sol.json and the certificates
     there)
```

```
#         python3 certify_line_pp.py <tag> <L*> [sweep*.jsonl pattern]      (picks the best sweep candidate for the target
     height L*)
import json, sys, os, subprocess, glob
from decimal import Decimal, ROUND_CEILING, localcontext
HERE = os.path.dirname(os.path.abspath(__file__))
def locate(name):   # repository layout (../hybrid_asymptotic/) or flat archive (same folder)
    for p in (os.path.join(HERE, name), os.path.join(HERE, '..', 'hybrid_asymptotic', name)):
        if os.path.exists(p): return p
    raise FileNotFoundError(name)

tag = sys.argv[1]; src = sys.argv[2]; pat = sys.argv[3] if len(sys.argv) > 3 else 'sweep*.jsonl'
if src.endswith('.json'): r = json.load(open(src))
else:
    L = float(src); rs = [json.loads(l) for f in glob.glob(pat) for l in open(f)]
    r = min((x for x in rs if x['Lstar'] == L), key=lambda x: x['val'])
D = 'line_'+tag; os.makedirs(D, exist_ok=True); sol = D+'/sol.json'; json.dump(r, open(sol, 'w'))
A1s = str(Decimal(repr(r['C1'])).quantize(Decimal('0.00001'), rounding=ROUND_CEILING))   # elem_gen.py checks A1 >= C1
     exactly
print('candidate', tag, 'd =', r['d'], 'C1 =', r['C1'], 'A3 (estimate) =', r['A3_est'], '-> A1 =', A1s, flush=True)

def run(cmd, log):
    with open(D+'/'+log, 'w') as fh: p = subprocess.run(cmd, stdout=fh, stderr=subprocess.STDOUT)
    out = open(D+'/'+log).read(); lines = out.strip().splitlines(); print(lines[-1] if lines else '(no output)', flush=True)
    return p.returncode, out
rc1, o1 = run(['python3', locate('certify_hybrid.py'), sol, '100'], 'cert_kernel.txt')
if not (rc1 == 0 and 'all certified' in o1): rc1, o1 = run(['python3', locate('certify_hybrid.py'), sol, '2000'], '
     cert_kernel.txt')
rc2, o2 = run(['python3', os.path.join(HERE, 'cz_gen_pp.py'), sol, '1e8', '1000'], 'cert_czeta_pp.txt')
rc3, o3 = run(['python3', os.path.join(HERE, 'elem_gen.py'), sol, A1s, '10000', 'N'], 'cert_elem.txt')
rc4, o4 = run(['python3', os.path.join(HERE, 'elem_gen.py'), sol, A1s, '10000', 'S'], 'cert_elem_S.txt')

def finish():
    # the certified upper bounds are decimal strings rounded upwards; add them in exact decimal arithmetic and round upwards
    with localcontext() as dc:
        dc.prec = 60
        cz = Decimal(json.load(open(sol.replace('.json', '_czpp.json')))['Czeta_upper'])
        fN = Decimal(json.load(open(sol.replace('.json', '_elem.json')))['Fsup_upper'])
        fS = Decimal(json.load(open(sol.replace('.json', '_elemS.json')))['Fsup_upper'])
        AN = (cz+fN).quantize(Decimal('0.001'), rounding=ROUND_CEILING); AS = (cz+fS).quantize(Decimal('0.001'), rounding=
     ROUND_CEILING)
    json.dump({'A1': A1s, 'A3_N': str(AN), 'A3_S': str(AS), 'Czeta_pp_upper': str(cz), 'Fsup_N': str(fN), 'Fsup_S': str(fS),
               'd': r['d'], 'B': r['B'], 'Lstar': r['Lstar']}, open(D+'/summary_pp.json', 'w'), indent=1)
    print('LINE %s CERTIFIED: |N(T)-main(T)| <= %s log T + %s ;  |S(T)| <= %s log T + %s   (T >= 1)' % (tag, A1s, AN, A1s, AS
     ), flush=True)
if rc1 == 0 and 'all certified' in o1 and rc2 == 0 and rc3 == 0 and rc4 == 0: finish()
else: print('LINE %s FAILED' % tag, flush=True); sys.exit(1)
```

S1.7. **Search programs.**

S1.8. `lpcommon.py: helpers of the search programs.`

```
# Floating-point helpers shared by the search programs (explore_discs.py, kernel_only.py, opt_line.py).
# Nothing here is used by the certification programs.
import numpy as np
from fractions import Fraction as Fr

def f(b, t, d):
    # f(b,t) = 2 atan((d+b)/t) + 2 atan((d-b)/t) - atan((2d+b)/t) - atan((2d-b)/t)
    return 2*np.arctan((b+d)/t)+2*np.arctan((d-b)/t)-np.arctan((b+2*d)/t)-np.arctan((2*d-b)/t)

def F(x, t, k, j):
    # F_Rj = Re (x+it)^-j,  F_Ij = -Im (x+it)^-j
    z = (x+1j*t)**(-j); return z.real if k == 'R' else -z.imag

def basis(m): return [('R', j) for j in range(1, m+1)]+[('I', j) for j in range(2, m+1)]

def tgrid(tmax, ngeom=700, nlin=400):
    # symmetric grid of heights t: geometric on [1e-4, tmax] plus a linear part on (0, 4], and a point near 0
    p = np.unique(np.concatenate([[1e-9], np.geomspace(1e-4, tmax, ngeom), np.linspace(0, 4, nlin)[1:]])); return np.
     concatenate([-p[::-1], p])

def bourgain_nodes():
    # piecewise linear profile through (lambda_j - kappa_j, kappa_j), (kappa_j, lambda_j) = A^j (13/84, 55/84), j = 0..7, and
      (1, 0)
    k, l = Fr(13, 84), Fr(55, 84); nodes = []
    for _ in range(8):
        nodes.append((l-k, k)); k, l = k/(2*k+2), Fr(1, 2)+l/(2*k+2)
    nodes.append((Fr(1), Fr(0))); return nodes

def J_float(nodes, c, R, n=40001):
```

```
    # (1/pi) int_0^pi e(c + R cos th) dth, e piecewise linear through nodes on [1/2, 1], e = 0 for sigma >= 1 (trapezoidal
     rule)
    xs = np.array([float(a) for a, _ in nodes]); es = np.array([float(e) for _, e in nodes])
    th = np.linspace(0, np.pi, n); s = c+R*np.cos(th); v = np.where(s >= 1, 0, np.interp(s, xs, es))
    return np.trapezoid(v, th)/np.pi
```

S1.9. **explore_discs.py: the hybrid linear programme.**

```
# Search program (not a certificate): the hybrid of the kernel majorant with Jensen discs.
# Linear programme in (c_iota, kappa_k):
#   minimise  c_R1/(2 pi) + (1/pi) sum_k kappa_k J_k
#   subject to  f(b,t) + eta/(1+t^2) <= U(b,t) + sum_k kappa_k W_k(b,t)  on a grid of (b,t), refined by cutting planes,
#               2 d c_R1 - 2 c_R2 >= eta  (limit of t^2 U at |t| -> oo),  kappa_k >= 0,
# with J_k = (1/pi) int_0^pi e(c_k + R_k cos th) dth for the profile e (PROF=P: e_P of profile_published.py; PROF=B: the
# Bourgain profile of lpcommon.bourgain_nodes).  Candidate discs: the family 'vwide' described in the article.
# Usage:  PROF=P python3 explore_discs.py <m> <d> [eta]      -> expl_m<m>_d<d>_<PROF>_eta<eta>.json (input of certify_hybrid.
     py)
import numpy as np, sys, os, json, time, math
from scipy.optimize import linprog, minimize
from scipy import sparse
from lpcommon import f, F, basis, tgrid, bourgain_nodes, J_float

PROF = os.environ.get('PROF', 'P')
if PROF == 'P':
    from profile_published import nodes_published; nodes = nodes_published()
elif PROF == 'B': nodes = bourgain_nodes()
else: raise SystemExit('PROF must be P or B')

def Wmat(discs, bb, tt):
    # sparse matrix of W_k(b,t) = log^+(R_k / sqrt((c_k - 1/2 - b)^2 + (t + v_k)^2))
    rows = []; cols = []; vals = []
    for k, (c, R, v) in enumerate(discs):
        dist = np.sqrt((c-0.5-bb)**2+(tt+v)**2)
        w = np.log(R/np.maximum(dist, 1e-300)); i = np.nonzero(w > 0)[0]
        rows.append(i); cols.append(np.full(len(i), k)); vals.append(w[i])
    r = np.concatenate(rows) if rows else np.zeros(0, int)
    return sparse.csr_matrix((np.concatenate(vals) if vals else [], (r, np.concatenate(cols) if cols else [])), shape=(len(bb
     ), len(discs)))
def Gmat(d, B, bb, tt): return np.array([F(d+bb, tt, k, j)+F(d-bb, tt, k, j) for (k, j) in B]).T

def solve(d, m, discs, eta=0.0, maxit=30, nb=26):
    B = basis(m); nB = len(B)
    sc = np.array([(d-0.5)**j for (_, j) in B])                 # scaling of the coefficients, for conditioning
    J = np.array([J_float(nodes, c, R) for c, R, v in discs])
    obj = np.concatenate([[sc[0]/(2*np.pi)], np.zeros(nB-1), J/np.pi])
    bb, tt = [a.ravel() for a in np.meshgrid(np.linspace(0, 0.5, nb), tgrid(300., 500, 300), indexing='ij')]
    def block(bb, tt): return sparse.hstack([sparse.csr_matrix(Gmat(d, B, bb, tt)*sc), Wmat(discs, bb, tt)]).tocsr(), f(bb,
     tt, d)+eta/(1+tt**2)
    A, Fv = block(bb, tt)
    arow = np.zeros(nB+len(discs)); arow[0] = 2*d*sc[0]; arow[1] = -2*sc[1]
    bf = np.linspace(0, 0.5, 101); tp = np.unique(np.concatenate([np.geomspace(1e-7, 3000., 6000), np.linspace(1e-4, 3.,
     30000)])); tf = np.concatenate([-tp[::-1], tp])
    def violation(x, used, du, bb, tt):   # f + margin - U - sum kappa W at the points (bb, tt); positive = violated
        v = f(bb, tt, d)+eta/(1+tt**2)-Gmat(d, B, bb, tt)@(x[:nB]*sc)
        if len(used): v = v-Wmat(du, bb, tt)@x[nB+used]
        return v
    for it in range(maxit):
        for method in ('highs', 'highs-ds', 'highs-ipm'):   # the dual simplex and the interior point method are tried if
     HiGHS' default fails
            r = linprog(obj, A_ub=-sparse.vstack([A, sparse.csr_matrix(arow)]), b_ub=-np.concatenate([Fv, [eta]]),
                        bounds=[(None, None)]*nB+[(0, None)]*len(discs), method=method)
            if r.status == 0: break
        assert r.status == 0, 'LP solver: '+r.message
        x = r.x; used = np.nonzero(x[nB:] > 1e-12)[0]; du = [discs[k] for k in used]
        nbl = []; ntl = []; worst = -1; cand = []
        for b in bf:   # violations on a finer grid -> new cutting planes (local maxima of the violation along t)
            v = violation(x, used, du, np.full_like(tf, b), tf)
            worst = max(worst, v.max()); idx = np.nonzero((v > 0) & (v >= np.roll(v, 1)) & (v >= np.roll(v, -1)))[0]
            nbl += [b]*len(idx); ntl += list(tf[idx])
            top = np.argsort(v)[-3:]; cand += [(v[i], b, tf[i]) for i in top]
        # the grid can miss narrow dips of U + sum kappa W - f (near the edges of the discs): refine the largest grid values
        # of the violation, including negative ones, by a local maximisation in (b, t), and add the maxima found as cutting
     planes
        cand.sort(reverse=True)
        for v0, b0, t0 in cand[:40]:
            if t0 == 0: continue
            g = lambda p: -violation(x, used, du, np.array([min(max(p[0], 0.0), 0.5)]), np.array([p[1] if p[1]*t0 > 0 else t0
     ]))[0]
            res = minimize(g, [b0, t0], method='Nelder-Mead', options={'xatol': 1e-7, 'fatol': 1e-12, 'maxiter': 400})
            bb_, tt_ = min(max(res.x[0], 0.0), 0.5), (res.x[1] if res.x[1]*t0 > 0 else t0)
            vv = -res.fun; worst = max(worst, vv)
```

```
            if vv > 0: nbl.append(bb_); ntl.append(tt_)
        print('  it %d  objective %.6f  worst violation %.2e  new cutting planes %d  rows %d' % (it, r.fun, worst, len(nbl),
     A.shape[0]), flush=True)
        if not nbl: break
        # new cutting planes, without near-duplicates (which make the linear programme ill-conditioned)
        pts = []
        for b_, t_ in zip(nbl, ntl):
            if all(abs(b_-b2) > 1e-6 or abs(t_-t2) > 1e-6*max(1.0, abs(t_)) for b2, t2 in pts): pts.append((b_, t_))
        A2, F2 = block(np.array([p[0] for p in pts]), np.array([p[1] for p in pts])); A = sparse.vstack([A, A2]).tocsr(); Fv
     = np.concatenate([Fv, F2])
    xs = x.copy(); xs[:nB] *= sc
    return r.fun, xs, B, J, worst, it

def family_vwide():
    # centres c, radii R = c - 1 + x with R <= c - 1/2, vertical offsets v
    C = [1.03, 1.06, 1.1, math.log(3), 1.15, 1.2, 1.25, 1.3, 1.4, 1.5, math.log(5), 2.0, math.e, 3.0]
    out = []
    for c in C:
        for R in [c-1+x for x in [0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5]]:
            if R <= 0 or R > c-0.5+1e-12: continue
            for v in [-2, -1.6, -1.3, -1.0, -0.8, -0.65, -0.5, -0.35, -0.2, 0, 0.2, 0.35, 0.5, 0.65, 0.8, 1.0, 1.3, 1.6, 2]:
                if abs(v) <= R+2.5: out.append((c, R, float(v)))
    return out

if __name__ == '__main__':
    m = int(sys.argv[1]); d = float(sys.argv[2]); eta = float(sys.argv[3]) if len(sys.argv) > 3 else 0.0
    discs = family_vwide(); t0 = time.time()
    val, x, B, J, w, it = solve(d, m, discs, eta)
    nB = len(B); keep = [k for k in range(len(discs)) if x[nB+k] > 1e-10]
    used_c = sorted(set(round(discs[k][0], 4) for k in keep))
    print('m=%d d=%.3f prof=%s eta=%g ndiscs=%d C(grid)=%.6f kernel=%.6f worst violation=%.2e it=%d used=%d centres=%s  %.0fs
     '
          % (m, d, PROF, eta, len(discs), val, x[0]/(2*np.pi), w, it, len(keep), used_c, time.time()-t0), flush=True)
    json.dump({'m': m, 'd': d, 'prof': PROF, 'eta': eta, 'B': B, 'c': list(x[:nB]), 'discs': [discs[k] for k in keep], 'kappa
     ': [x[nB+k] for k in keep],
               'J_grid': [J[k] for k in keep], 'C_grid_objective': val}, open('expl_m%d_d%.2f_%s_eta%g.json' % (m, d, PROF,
     eta), 'w'))
```

S1.10. kernel_only.py: kernel-only optima (Remark on the infimum).

```
# Search program (not a certificate): the smallest leading constant c_R1/(2 pi) of a kernel majorant without discs,
#   minimise c_R1/(2 pi)  subject to  f(b,t) <= U(b,t) on a grid of (b,t), refined by cutting planes, and 2 d c_R1 - 2 c_R2
     >= 0,
# for the basis of order m and the given d; with the option 'rh' the constraint is imposed only at b = 0 (beta = 1/2), which
     is
# what would be needed if all zeros were on the critical line.  Used for Remark "On the infimum of the leading constant".
# Usage:  python3 kernel_only.py <m> <d1,d2,...> [rh]
import numpy as np, sys, time
from scipy.optimize import linprog
from lpcommon import f, F, basis, tgrid

def Gmat(d, B, bb, tt): return np.array([F(d+bb, tt, k, j)+F(d-bb, tt, k, j) for (k, j) in B]).T

def solve(d, m, rh=False, maxit=60, nb=26, tol=1e-9):
    B = basis(m); nB = len(B)
    sc = np.array([(d-0.5)**j for (_, j) in B])                # scaling of the coefficients, for conditioning
    obj = np.zeros(nB); obj[0] = sc[0]/(2*np.pi)
    bgrid = np.array([0.0]) if rh else np.linspace(0, 0.5, nb)
    bb, tt = [a.ravel() for a in np.meshgrid(bgrid, tgrid(300.), indexing='ij')]
    A = Gmat(d, B, bb, tt)*sc; Fv = f(bb, tt, d)
    arow = np.zeros(nB); arow[0] = 2*d*sc[0]; arow[1] = -2*sc[1]
    bf = np.array([0.0]) if rh else np.linspace(0, 0.5, 201); tp = np.geomspace(1e-8, 3000., 12000); tf = np.concatenate([-tp
     [::-1], tp])
    for it in range(maxit):
        r = linprog(obj, A_ub=-np.vstack([A, arow]), b_ub=-np.concatenate([Fv, [0.0]]), bounds=[(None, None)]*nB, method='
     highs')
        assert r.status == 0, 'LP solver: '+r.message
        x = r.x; nbl = []; ntl = []; worst = -1
        for b in bf:   # violations on a finer grid -> new cutting planes (local maxima of the violation)
            v = f(b, tf, d)-Gmat(d, B, np.full_like(tf, b), tf)@(x*sc)
            worst = max(worst, v.max()); idx = np.nonzero((v > tol) & (v >= np.roll(v, 1)) & (v >= np.roll(v, -1)))[0]
            nbl += [b]*len(idx); ntl += list(tf[idx])
        if not nbl: break
        A = np.vstack([A, Gmat(d, B, np.array(nbl), np.array(ntl))*sc]); Fv = np.concatenate([Fv, f(np.array(nbl), np.array(
     ntl), d)])
    return r.fun, worst, it

if __name__ == '__main__':
    m = int(sys.argv[1]); ds = [float(x) for x in sys.argv[2].split(',')]; rh = len(sys.argv) > 3 and sys.argv[3] == 'rh'
    for d in ds:
        t0 = time.time(); val, w, it = solve(d, m, rh)
```

```
        print('m=%d d=%.2f %s C1(grid)=%.5f worst violation=%.1e it=%d  %.0fs' % (m, d, 'constraint at beta=1/2 only' if rh
      else 'all beta', val, w, it, time.time()-t0), flush=True)
```

S1.11. opt_line.py: the second-order cone programme of one explicit line.

```
# Search program (not a certificate): optimise one explicit line |N(T)-main(T)| <= A1 log T + A3 for a target height L* = log
      T*.
# Kernel U = sum c_iota G_iota (basis R1..R_mR, I2..I_mI), d fixed. Convex programme (second-order cone, Clarabel through
     cvxpy):
#   minimise  (c_R1/2pi) L* + C_zeta(c) + z
#   s.t.      f(b,t) + eta/(1+t^2) <= U(b,t)      on a (b,t) grid, refined by cutting planes
#             2d c_R1 - 2 c_R2 >= eta              (limit of t^2 U at |t| -> oo)
#             z >= F_u(T;c), z >= F_l(T;c)         on a T grid in [1,1e6], and z >= the limit at T = oo
# C_zeta(c) = sum_n Lambda(n) n^{-sigma1} || (a(log n), b(log n)) ||_2  (n <= n0) + quadrature of the tail; with PPMAX > 0
     the
# primes p <= PPMAX enter through the maximum of f_p over a grid of 96 phases (per-prime phase maximum).
# Usage:  [PPMAX=300] python3 opt_line.py <d> <mR> <mI> <L*> <out.json>
# Output: JSON with the coefficients as decimal strings that round-trip to the binary floats, the input of certify_line_pp.py
     .
import sys, json, math, time, os, numpy as np, cvxpy as cp, mpmath as mp
PPMAX = int(os.environ.get('PPMAX', '0'))   # per-prime phase maximum in the objective for the primes p <= PPMAX (0: none)
sys.path.append(os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'hybrid_asymptotic'))   # lpcommon.py lives
     there (or next to this file in the flat archive)
from lpcommon import f, F, tgrid

def basis(mR, mI): return [('R', j) for j in range(1, mR+1)] + [('I', j) for j in range(2, mI+1)]

def kernel_rows(d, B, bb, tt):
    G = np.array([F(d+bb, tt, k, j) + F(d-bb, tt, k, j) for (k, j) in B]).T
    return G, f(bb, tt, d)

def primes(N):
    s = np.ones(N+1, bool); s[:2] = False
    for i in range(2, int(N**0.5)+1):
        if s[i]: s[i*i::i] = False
    return np.nonzero(s)[0]

def czeta_data(d, B, n0):
    # rows (one per prime power n <= n0, then one per quadrature node of the tail) such that
    # C_zeta ~ sum_rows w * || (rows_a @ c, rows_b @ c + beta0) ||_2;  prime_of[row] = p for the row of n = p^k, 0 for the
     tail
    s1 = 0.5+d; rows_a = []; rows_b = []; beta0 = []; w = []; prime_of = []
    def add(L, wt, p):
        rows_a.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'R' else 0 for (kk, j) in B])
        rows_b.append([-(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'I' else 0 for (kk, j) in B])
        beta0.append((math.exp(-d*L)-2)/(L*math.pi)); w.append(wt); prime_of.append(p)
    for p in primes(n0):
        lp = math.log(p); n = p; k = 1
        while n <= n0:
            add(k*lp, lp*math.exp(-s1*k*lp), int(p)); n *= p; k += 1
    # tail n > n0: psi(x) ~ x, sum Lambda(n) g(n) ~ int g(x) dx = int g e^L dL ; Gauss-Legendre in L on [log n0, 700]
    xg, wg = np.polynomial.legendre.leggauss(400); a, b = math.log(n0), 700.0
    for x, ww in zip(xg, wg):
        L = (b-a)/2*x+(a+b)/2; add(L, (b-a)/2*ww*math.exp((1-s1)*L), 0)
    return np.array(rows_a), np.array(rows_b), np.array(beta0), np.array(w), np.array(prime_of)


def czeta_pp_data(d, B, PP=300, n0=20000):
    """for primes p <= PP: list of (kvec, wvec, RAk, RBk, b0k) to build max_x f_p(x) on a grid; other n via hypot rows."""
    s1 = 0.5+d; groups = []
    for p in primes(PP):
        lp = math.log(p); n = p; k = 1; ks = []; ra = []; rb = []; bz = []
        while n <= n0:
            L = k*lp; w = lp*math.exp(-s1*L)
            ra.append([-w*(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'R' else 0 for (kk, j) in B])
            rb.append([-w*(L**(j-1)/math.factorial(j-1))/math.pi if kk == 'I' else 0 for (kk, j) in B])
            bz.append(w*(math.exp(-d*L)-2)/(L*math.pi)); ks.append(k); n *= p; k += 1
        groups.append((p, np.array(ks), np.array(ra), np.array(rb), np.array(bz)))
    return groups

def elem_data(d, B, Tgrid):
    mp.mp.dps = 25
    s1r = mp.mpf(1)/2+d; s2r = mp.mpf(1)/2+2*d
    base = []; Phi = []
    for T in Tgrid:
        T = mp.mpf(T); s1 = s1r+1j*T; s2 = s2r+1j*T
        Ael = lambda s: mp.im(mp.log(s)+mp.log(s-1))-T/2*mp.log(mp.pi)+mp.im(mp.loggamma(s/2))
        base.append(float((2*Ael(s1)-Ael(s2))/mp.pi-T/(2*mp.pi)*mp.log(T/(2*mp.pi*mp.e))))
        row = []
        for (k, j) in B:
            if j == 1: D = 1/s1+1/(s1-1)-mp.log(mp.pi)/2+mp.psi(0, s1/2)/2
```

```
            else: D = s1**(-j)+(s1-1)**(-j)+(-1)**(j-1)/mp.factorial(j-1)/2**j*mp.psi(j-1, s1/2)
            row.append(float(mp.re(D)) if k == 'R' else float(-mp.im(D)))
        Phi.append(row)
    return np.array(base), np.array(Phi)

def optimize(d, mR, mI, Lstar, eta=2e-4, n0=20000, maxit=25, verbose=True):
    B = basis(mR, mI); nB = len(B); i1 = B.index(('R', 1)); i2 = B.index(('R', 2))
    isR = np.array([k == 'R' for k, _ in B])
    bb, tt = [a.ravel() for a in np.meshgrid(np.linspace(0, 0.5, 26), tgrid(300.), indexing='ij')]
    G, Fv = kernel_rows(d, B, bb, tt); Fv = Fv+eta/(1+tt**2)
    RA, RB, b0, w, prime_of = czeta_data(d, B, n0)
    if PPMAX:
        GR = czeta_pp_data(d, B, PPMAX, n0)
        maskPP = np.array([not (p and p <= PPMAX) for p in prime_of])     # rows of the prime powers p^k with p <= PPMAX are
     replaced by the phase maximum
        assert (~maskPP).sum() == sum(len(g[1]) for g in GR)
    Tg = np.unique(np.concatenate([np.geomspace(1, 1e6, 260), [1.0]]))
    base, Phi = elem_data(d, B, Tg); logT = np.log(Tg)
    sc = np.array([(d-0.5)**j for (_, j) in B])        # scaling for conditioning
    bfine = np.linspace(0, 0.5, 201); tp = np.geomspace(1e-8, 3000., 12000); tfine = np.concatenate([-tp[::-1], tp])
    for it in range(maxit):
        y = cp.Variable(nB); z = cp.Variable(); c = cp.multiply(sc, y)
        cons = [G@c >= Fv, 2*d*c[i1]-2*c[i2] >= eta]
        sign = np.where(isR, 1.0, -1.0)
        Fu = base+(Phi@c)/math.pi-c[i1]/(2*math.pi)*logT
        Fl = -base+(Phi@cp.multiply(sign, c))/math.pi-c[i1]/(2*math.pi)*logT
        cons += [z >= Fu, z >= Fl, z >= 7/8-c[i1]/(2*math.pi)*math.log(2*math.pi)]
        if PPMAX:
            xg = np.linspace(0, 2*np.pi, 96, endpoint=False); tp_ = cp.Variable(len(GR))
            for gi, (p, ks, ra, rb, bz) in enumerate(GR):
                Cm = np.cos(np.outer(xg, ks)); Sm = np.sin(np.outer(xg, ks))
                cons.append(tp_[gi] >= Cm@(ra@c)+Sm@(rb@c+bz))
            Cz = cp.sum(tp_)+cp.sum(cp.multiply(w[maskPP], cp.norm(cp.vstack([RA[maskPP]@c, RB[maskPP]@c+b0[maskPP]]), 2,
     axis=0)))
        else:
            Cz = cp.sum(cp.multiply(w, cp.norm(cp.vstack([RA@c, RB@c+b0]), 2, axis=0)))
        obj = c[i1]/(2*math.pi)*Lstar+Cz+z
        prob = cp.Problem(cp.Minimize(obj), cons); prob.solve(solver=cp.CLARABEL)
        assert prob.status in (cp.OPTIMAL, cp.OPTIMAL_INACCURATE), 'solver status: '+str(prob.status)
        cv = sc*y.value
        nb = []; nt = []; worst = -1
        for b in bfine:
            Gr, Fr = kernel_rows(d, B, np.full_like(tfine, b), tfine); v = Fr+eta/(1+tfine**2)-Gr@cv; worst = max(worst, v.
     max())
            idx = np.where((v > 0) & (v >= np.roll(v, 1)) & (v >= np.roll(v, -1)))[0]; nb += [b]*len(idx); nt += list(tfine[
     idx])
        if verbose: print('  it %d obj=%.5f C1=%.5f Cz~%.4f z=%.4f worst=%.2e' % (it, prob.value, cv[i1]/(2*math.pi), Cz.
     value, z.value, worst), flush=True)
        if not nb: break
        G2, F2 = kernel_rows(d, B, np.array(nb), np.array(nt)); G = np.vstack([G, G2]); Fv = np.concatenate([Fv, F2+eta/(1+np
     .array(nt)**2)])
    return dict(d=d, B=B, c=list(map(float, cv)), C1=float(cv[i1]/(2*math.pi)), Cz_est=float(Cz.value), z_est=float(z.value),
                A3_est=float(Cz.value+z.value), Lstar=Lstar, eta=eta, PPMAX=PPMAX, discs=[], kappa=[], m=max(j for _, j in B)
     )

if __name__ == '__main__':
    d = float(sys.argv[1]); mR = int(sys.argv[2]); mI = int(sys.argv[3]); Ls = float(sys.argv[4]); out = sys.argv[5]
    t0 = time.time(); r = optimize(d, mR, mI, Ls)
    print('RESULT d=%.3f mR=%d mI=%d L*=%g  A1=%.5f A3~%.4f  value@L*~%.4f  (%.0fs)' % (d, mR, mI, Ls, r['C1'], r['A3_est'],
     r['C1']*Ls+r['A3_est'], time.time()-t0), flush=True)
    json.dump(r, open(out, 'w'))
```

S2. Outputs of the certification programs

Each output begins with the SHA-256 digests of its input file and of the programs that produced it. Only the plain decimal numbers printed after `<` or `<=` (without a radius) are produced by directed decimal rounding (`up_str` of `certutil.py`) and are asserted as bounds; every display of the form `[m +/- r]` is a ball printed with the standard `str` of `python-flint`, which rounds the midpoint to nearest, and is diagnostic only (for the final value of C_{H} the certificate also prints its radius, as a ball enclosing the radius, and a twenty-digit enclosure obtained by directed rounding). The running times printed at the end of the lines are informative only and depend on the machine; they are not part of the certified data. There are $1+6\cdot 4=25$ certificates: one for the hybrid inequality and four (kernel inequality, C_ζ, elementary part in modes N and S) for each of the six lines.

S2.1. **Hybrid: `certify_hybrid.py hyb_m10_P.json 40`.**

```
hyb_m10_P.json sha256 2b94bf5d689eee6e5688699be67d4725255aa97453d9115dcf2010b6b0e6a9fc
certify_hybrid.py sha256 fdfe7462c1b017d4167aa7199a16a82117c74fd1a1fbb0ed5dbcc69b210402a5
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
profile_published.py sha256 06a554e573fabe71b6049e704349df53cad774712423042241436500acadb4bc
python 3.13.18, python-flint 0.9.0, precision 256 bits
d = 9/10  m = 10  discs used: 90  profile e_P (profile_published.py)
C = c_R1/(2pi) + sum kappa_k J_k/pi = [0.074859614271090718755015645 +/- 3.78e-27]  (ball, midpoint and radius)
   radius of the ball: [2.94e-76 +/- 2.74e-79]   enclosure of C with directed rounding: [0.07485961427109071875,
     0.07485961427109071876]
kernel part c_R1/(2pi) = [0.065582182784544440363 +/- 1.95e-22]
limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 = [0.001408437209 +/- 3.24e-13]  positive: True
t in (0,T1]: certified, boxes 2028149
t in [-T1,0): certified, boxes 2294869
|t|>=T1: certified, boxes 277098
all certified;  C < 0.0748596142711   time 837s
```

S2.2. **Line ℓ_5: kernel inequality.**

```
sol.json sha256 febfa49528729b67e8af3b65920a6682545be70e6ec46b9a5262448e9e946574
certify_hybrid.py sha256 fdfe7462c1b017d4167aa7199a16a82117c74fd1a1fbb0ed5dbcc69b210402a5
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
d = 1  m = 5  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = [0.20175655042691417703820886 +/- 4.65e-26]  (ball, midpoint and radius)
   radius of the ball: [3.27e-78 +/- 1.70e-81]   enclosure of C with directed rounding: [0.20175655042691417703,
     0.20175655042691417704]
kernel part c_R1/(2pi) = [0.20175655042691417704 +/- 1.72e-21]
limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 = [1.133488684 +/- 3.74e-10]  positive: True
t in (0,T1]: certified, boxes 10255
t in [-T1,0): certified, boxes 4207
|t|>=T1: certified, boxes 2
all certified;  C < 0.201756550427   time 1s
```

S2.3. **Line ℓ_5: C_ζ.**

```
sol.json sha256 febfa49528729b67e8af3b65920a6682545be70e6ec46b9a5262448e9e946574
cz_gen_pp.py sha256 a16e89acdcb472cf121ae6e212f9399c774ac2fefdf731a69e919545d763ee02
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
per-prime phase maximum for p <= 1000 : gain [0.13567339 +/- 3.39e-9]
primes 5761455  partial sum (n<=100000000) = [0.700266525818397 +/- 2.62e-16]  psi(N0) = [99998242.7966268 +/- 1.77e-8]  51s
g monotone on [2000, oo): crude bound [+/- 4.40e-3] < sigma1
tail <= [0.003333488646 +/- 2.02e-13]  (int_{N0}^{e^LB} G = [0.003174276158 +/- 3.32e-14] , int_{e^L1}^{e^LB} g = [+/- 3.23e
     -163] , far tail <= [2.6758e-167 +/- 3.15e-172] )
C_zeta < 0.703600014465    time 52s
```

S2.4. **Line ℓ_5: elementary part, mode N.**

```
sol.json sha256 febfa49528729b67e8af3b65920a6682545be70e6ec46b9a5262448e9e946574
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= 0.6653643978   sup F_l <= -0.7495433723
T>=T0: F_u <= 0.5045914644  F_l <= -1.245408535  (K = [3.95201 +/- 3.63e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.665364397774    time 60s
```

S2.5. **Line ℓ_5: elementary part, mode S.**

```
sol.json sha256 febfa49528729b67e8af3b65920a6682545be70e6ec46b9a5262448e9e946574
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= -0.2236696656   sup F_l <= 0.1394908934
T>=T0: F_u <= -0.3703919570  F_l <= -0.3703919570  (K = [4.11780 +/- 2.69e-8] )
SUP over T>=1 of max(F_u,F_l) <= 0.139490893363    time 61s
```

S2.6. **Line ℓ_{20}: kernel inequality.**

```
sol.json sha256 a95801710718f4fdbfb80f2f66ece1d071a440f5650c2a2ae2700fc13032143f
certify_hybrid.py sha256 fdfe7462c1b017d4167aa7199a16a82117c74fd1a1fbb0ed5dbcc69b210402a5
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
d = 6/5  m = 5  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = [0.10822193548420830107702960 +/- 1.10e-26]  (ball, midpoint and radius)
   radius of the ball: [1.67e-78 +/- 4.53e-81]   enclosure of C with directed rounding: [0.10822193548420830107,
     0.10822193548420830108]
kernel part c_R1/(2pi) = [0.10822193548420830108 +/- 2.98e-21]
limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 = [0.02372706155 +/- 2.76e-12]  positive: True
t in (0,T1]: certified, boxes 473575
t in [-T1,0): certified, boxes 279773
|t|>=T1: certified, boxes 1022
```

```
all certified;  C < 0.108221935485   time 46s
```

S2.7. **Line ℓ_{20}: C_ζ.**

```
sol.json sha256 a95801710718f4fdbfb80f2f66ece1d071a440f5650c2a2ae2700fc13032143f
cz_gen_pp.py sha256 a16e89acdcb472cf121ae6e212f9399c774ac2fefdf731a69e919545d763ee02
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
per-prime phase maximum for p <= 1000 : gain [0.21171880 +/- 3.25e-9]
primes 5761455  partial sum (n<=100000000) = [1.46785649757579 +/- 4.11e-15]  psi(N0) = [99998242.7966268 +/- 1.77e-8]  53s
g monotone on [2000, oo): crude bound [+/- 0.0145] < sigma1
tail <= [0.0002188390453 +/- 1.36e-15]  (int_{N0}^{e^LB} G = [0.0002060161219 +/- 1.10e-14] , int_{e^L1}^{e^LB} g = [+/- 7.27
     e-234] , far tail <= [7.2489e-238 +/- 1.12e-244] )
C_zeta < 1.46807533663    time 53s
```

S2.8. **Line ℓ_{20}: elementary part, mode N.**

```
sol.json sha256 a95801710718f4fdbfb80f2f66ece1d071a440f5650c2a2ae2700fc13032143f
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= 0.7020707094   sup F_l <= -1.072389149
T>=T0: F_u <= 0.6766231016  F_l <= -1.073376898  (K = [5.21715 +/- 1.54e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.702070709399    time 61s
```

S2.9. **Line ℓ_{20}: elementary part, mode S.**

```
sol.json sha256 a95801710718f4fdbfb80f2f66ece1d071a440f5650c2a2ae2700fc13032143f
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= -0.1827864943   sup F_l <= -0.1965944846
T>=T0: F_u <= -0.1983603198  F_l <= -0.1983603198  (K = [5.38293 +/- 4.87e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.182786494342    time 62s
```

S2.10. **Line ℓ_{60}: kernel inequality.**

```
sol.json sha256 79d3366fe78fe865de0b9dbd01bab13c6d5380a91913b78c2f4da095fb389acf
certify_hybrid.py sha256 fdfe7462c1b017d4167aa7199a16a82117c74fd1a1fbb0ed5dbcc69b210402a5
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
d = 1  m = 5  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = [0.094721574236353990058 50854 +/- 1.28e-27]  (ball, midpoint and radius)
   radius of the ball: [1.60e-78 +/- 3.05e-82]   enclosure of C with directed rounding: [0.094721574236353990058,
     0.094721574236353990059]
kernel part c_R1/(2pi) = [0.094721574236353990059 +/- 4.92e-22]
limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 = [0.008043907939 +/- 4.87e-13]  positive: True
t in (0,T1]: certified, boxes 1441685
t in [-T1,0): certified, boxes 483889
|t|>=T1: certified, boxes 4434
all certified;  C < 0.0947215742364   time 110s
```

S2.11. **Line ℓ_{60}: C_ζ.**

```
sol.json sha256 79d3366fe78fe865de0b9dbd01bab13c6d5380a91913b78c2f4da095fb389acf
cz_gen_pp.py sha256 a16e89acdcb472cf121ae6e212f9399c774ac2fefdf731a69e919545d763ee02
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
per-prime phase maximum for p <= 1000 : gain [0.24105429 +/- 2.19e-10]
primes 5761455  partial sum (n<=100000000) = [2.09743955289942 +/- 3.84e-15]  psi(N0) = [99998242.7966268 +/- 1.77e-8]  52s
g monotone on [2000, oo): crude bound [+/- 0.0149] < sigma1
tail <= [0.006829934351 +/- 6.03e-14]  (int_{N0}^{e^LB} G = [0.006476069071 +/- 3.90e-13] , int_{e^L1}^{e^LB} g = [+/- 1.23e
     -164] , far tail <= [1.7143e-168 +/- 4.48e-173] )
C_zeta < 2.10426948726    time 53s
```

S2.12. **Line ℓ_{60}: elementary part, mode N.**

```
sol.json sha256 79d3366fe78fe865de0b9dbd01bab13c6d5380a91913b78c2f4da095fb389acf
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= 0.7289415578   sup F_l <= -1.047579458
T>=T0: F_u <= 0.7013136727  F_l <= -1.048686327  (K = [4.00282 +/- 3.70e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.728941557753    time 61s
```

S2.13. **Line ℓ_{60}: elementary part, mode S.**

```
sol.json sha256 79d3366fe78fe865de0b9dbd01bab13c6d5380a91913b78c2f4da095fb389acf
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= -0.1584757589   sup F_l <= -0.1717847946
T>=T0: F_u <= -0.1736697487  F_l <= -0.1736697487  (K = [4.16860 +/- 2.71e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.158475758991    time 62s
```

S2.14. **Line ℓ_{150}: kernel inequality.**

```
sol.json sha256 88654e2339e4f68925dd36bb344e3ed3ea22a373bdc8fa02dec09407e7c066ea
certify_hybrid.py sha256 fdfe7462c1b017d4167aa7199a16a82117c74fd1a1fbb0ed5dbcc69b210402a5
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
d = 17/20  m = 5  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = [0.093480628436037879597919471 +/- 2.87e-27]  (ball, midpoint and radius)
   radius of the ball: [1.59e-78 +/- 3.49e-81]   enclosure of C with directed rounding: [0.093480628436037879597,
     0.093480628436037879598]
kernel part c_R1/(2pi) = [0.093480628436037879597 +/- 1.95e-22]
limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 = [0.007450671016 +/- 2.69e-13]  positive: True
t in (0,T1]: certified, boxes 1658655
t in [-T1,0): certified, boxes 353921
|t|>=T1: certified, boxes 3198
all certified;  C < 0.0934806284361   time 123s
```

S2.15. **Line ℓ_{150}: C_ζ.**

```
sol.json sha256 88654e2339e4f68925dd36bb344e3ed3ea22a373bdc8fa02dec09407e7c066ea
cz_gen_pp.py sha256 a16e89acdcb472cf121ae6e212f9399c774ac2fefdf731a69e919545d763ee02
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
per-prime phase maximum for p <= 1000 : gain [0.25085637 +/- 3.02e-9]
primes 5761455  partial sum (n<=100000000) = [2.21455819652827 +/- 4.70e-15]  psi(N0) = [99998242.7966268 +/- 1.77e-8]  53s
g monotone on [2000, oo): crude bound [+/- 7.01e-3] < sigma1
tail <= [0.04122525901 +/- 2.13e-12]  (int_{N0}^{e^LB} G = [0.03953310421 +/- 1.61e-12] , int_{e^L1}^{e^LB} g = [+/- 2.53e
     -111] , far tail <= [3.6826e-115 +/- 3.45e-120] )
C_zeta < 2.25578345555    time 53s
```

S2.16. **Line ℓ_{150}: elementary part, mode N.**

```
sol.json sha256 88654e2339e4f68925dd36bb344e3ed3ea22a373bdc8fa02dec09407e7c066ea
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= 0.7303168470   sup F_l <= -1.045307483
T>=T0: F_u <= 0.7035135754  F_l <= -1.046486424  (K = [3.19479 +/- 4.84e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.730316846936    time 61s
```

S2.17. **Line ℓ_{150}: elementary part, mode S.**

```
sol.json sha256 88654e2339e4f68925dd36bb344e3ed3ea22a373bdc8fa02dec09407e7c066ea
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= -0.1575763483   sup F_l <= -0.1695128192
T>=T0: F_u <= -0.1714698460  F_l <= -0.1714698460  (K = [3.36057 +/- 1.57e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.157576348369    time 62s
```

S2.18. **Line ℓ_{400}: kernel inequality.**

```
sol.json sha256 961b299dafd10eca996a4a542d3f9f6b3afa729cd7bbde4fd11d5759c1ab42c8
certify_hybrid.py sha256 fdfe7462c1b017d4167aa7199a16a82117c74fd1a1fbb0ed5dbcc69b210402a5
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
d = 17/20  m = 10  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = [0.083986018406625982801154790 +/- 4.65e-27]  (ball, midpoint and radius)
   radius of the ball: [1.54e-78 +/- 1.28e-81]   enclosure of C with directed rounding: [0.083986018406625982801,
     0.083986018406625982802]
kernel part c_R1/(2pi) = [0.083986018406625982801 +/- 1.55e-22]
limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 = [0.0002002739955 +/- 2.69e-14]  positive: True
t in (0,T1]: certified, boxes 11835193
t in [-T1,0): certified, boxes 9801985
|t|>=T1: certified, boxes 4194302
all certified;  C < 0.0839860184067   time 2316s
```

S2.19. **Line ℓ_{400}: C_ζ.**

```
sol.json sha256 961b299dafd10eca996a4a542d3f9f6b3afa729cd7bbde4fd11d5759c1ab42c8
cz_gen_pp.py sha256 a16e89acdcb472cf121ae6e212f9399c774ac2fefdf731a69e919545d763ee02
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
per-prime phase maximum for p <= 1000 : gain [0.28179774 +/- 3.28e-9]
primes 5761455  partial sum (n<=100000000) = [4.01127229118254 +/- 1.94e-15]  psi(N0) = [99998242.7966268 +/- 1.77e-8]  80s
g monotone on [5000, oo): crude bound [+/- 0.0114] < sigma1
tail <= [0.4524693305 +/- 1.73e-11]  (int_{N0}^{e^LB} G = [0.4325160730 +/- 4.01e-11] , int_{e^L1}^{e^LB} g = [+/- 6.02e-102]
      , far tail <= [2.8606e-111 +/- 3.11e-116] )
C_zeta < 4.46374162170    time 81s
```

S2.20. **Line ℓ_{400}: elementary part, mode N.**

```
sol.json sha256 961b299dafd10eca996a4a542d3f9f6b3afa729cd7bbde4fd11d5759c1ab42c8
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= 0.7375655061   sup F_l <= -1.027807128
T>=T0: F_u <= 0.7209635502  F_l <= -1.029036449  (K = [3.19527 +/- 2.94e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.737565506071    time 79s
```

S2.21. **Line ℓ_{400}: elementary part, mode S.**

```
sol.json sha256 961b299dafd10eca996a4a542d3f9f6b3afa729cd7bbde4fd11d5759c1ab42c8
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= -0.1458267316   sup F_l <= -0.1520124644
T>=T0: F_u <= -0.1540198711  F_l <= -0.1540198711  (K = [3.36106 +/- 6.67e-7] )
SUP over T>=1 of max(F_u,F_l) <= -0.145826731639    time 81s
```

S2.22. **Line ℓ_{6000}: kernel inequality.**

```
sol.json sha256 bc61062c39431f9498808273a0673ca1381c0a16313f88d89d2684e53da8431e
certify_hybrid.py sha256 fdfe7462c1b017d4167aa7199a16a82117c74fd1a1fbb0ed5dbcc69b210402a5
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
d = 17/20  m = 10  discs used: 0
C = c_R1/(2pi) + sum kappa_k J_k/pi = [0.083814170602092884236 29767 +/- 2.14e-27]  (ball, midpoint and radius)
   radius of the ball: [1.54e-78 +/- 3.35e-82]   enclosure of C with directed rounding: [0.083814170602092884236,
     0.083814170602092884237]
kernel part c_R1/(2pi) = [0.083814170602092884236 +/- 2.98e-22]
limit of t^2 U at |t| -> oo = 2 d c_R1 - 2 c_R2 = [0.0002027460239 +/- 3.01e-14]  positive: True
t in (0,T1]: certified, boxes 14760921
t in [-T1,0): certified, boxes 15866017
|t|>=T1: certified, boxes 4194302
all certified;  C < 0.0838141706021   time 3125s
```

S2.23. **Line ℓ_{6000}: C_ζ.**

```
sol.json sha256 bc61062c39431f9498808273a0673ca1381c0a16313f88d89d2684e53da8431e
cz_gen_pp.py sha256 a16e89acdcb472cf121ae6e212f9399c774ac2fefdf731a69e919545d763ee02
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
per-prime phase maximum for p <= 1000 : gain [0.28466228 +/- 4.66e-10]
primes 5761455  partial sum (n<=100000000) = [4.10087443545128 +/- 2.45e-15]  psi(N0) = [99998242.7966268 +/- 1.77e-8]  80s
g monotone on [5000, oo): crude bound [+/- 0.0108] < sigma1
tail <= [0.4886359120 +/- 3.99e-12]  (int_{N0}^{e^LB} G = [0.4671146906 +/- 3.15e-11] , int_{e^L1}^{e^LB} g = [+/- 6.36e-102]
      , far tail <= [2.9139e-111 +/- 4.83e-116] )
C_zeta < 4.58951034745    time 80s
```

S2.24. **Line ℓ_{6000}: elementary part, mode N.**

```
sol.json sha256 bc61062c39431f9498808273a0673ca1381c0a16313f88d89d2684e53da8431e
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= 0.7357375257   sup F_l <= -1.027508246
T>=T0: F_u <= 0.7212794332  F_l <= -1.028720566  (K = [3.19575 +/- 1.13e-6] )
SUP over T>=1 of max(F_u,F_l) <= 0.735737525685    time 80s
```

S2.25. **Line ℓ_{6000}: elementary part, mode S.**

```
sol.json sha256 bc61062c39431f9498808273a0673ca1381c0a16313f88d89d2684e53da8431e
elem_gen.py sha256 25530a892cba6b7ef5c5c229da418d28a7e86622f11552a8f7abb963a6093670
certutil.py sha256 47322401572e98fab93924be05e7172d9d88a80685ee2818973a1081430e3220
python 3.13.18, python-flint 0.9.0, precision 256 bits
intervals 9215  sup F_u on [1,T0] <= -0.1464103660   sup F_l <= -0.1517135821
T>=T0: F_u <= -0.1537039882  F_l <= -0.1537039882  (K = [3.36154 +/- 2.48e-6] )
SUP over T>=1 of max(F_u,F_l) <= -0.146410366017    time 80s
```

S2.26. **Grid optima of `kernel_only.py` (Remark 2.5 of the article; not certified).** Lines marked `NOT CONVERGED` are runs whose cutting-plane iteration stopped with a residual violation $\geq 10^{-5}$ on the fine grid; they are not quoted in the text. The marks are added here; the file `kernel_only.txt` is unchanged.

```
m=4 d=0.70 all beta C1(grid)=0.08831 worst violation=1.8e-08 it=59  120s
m=4 d=0.80 all beta C1(grid)=0.08894 worst violation=3.0e-13 it=9  20s
m=4 d=0.90 all beta C1(grid)=0.09061 worst violation=1.5e-10 it=3  8s
m=10 d=0.70 all beta C1(grid)=0.08160 worst violation=1.4e-13 it=5  33s
m=10 d=0.80 all beta C1(grid)=0.08071 worst violation=7.4e-08 it=59  233s
m=10 d=0.90 all beta C1(grid)=0.08100 worst violation=8.6e-09 it=59  173s
m=20 d=0.70 all beta C1(grid)=0.07994 worst violation=3.1e-09 it=59  373s
m=20 d=0.80 all beta C1(grid)=0.07983 worst violation=1.6e-09 it=59  378s
m=20 d=0.90 all beta C1(grid)=0.07996 worst violation=1.9e-06 it=59  382s
m=40 d=0.70 all beta C1(grid)=0.07978 worst violation=2.1e-02 it=59  2057s   # NOT CONVERGED (not quoted)
m=4 d=0.70 constraint at beta=1/2 only C1(grid)=0.04375 worst violation=3.3e-16 it=3  0s
m=4 d=0.80 constraint at beta=1/2 only C1(grid)=0.05000 worst violation=1.6e-12 it=2  0s
m=4 d=0.90 constraint at beta=1/2 only C1(grid)=0.05625 worst violation=2.4e-12 it=2  0s
m=10 d=0.70 constraint at beta=1/2 only C1(grid)=0.01747 worst violation=3.3e-04 it=59  2s   # NOT CONVERGED (not quoted)
m=10 d=0.80 constraint at beta=1/2 only C1(grid)=0.02002 worst violation=3.5e-14 it=3  0s
m=10 d=0.90 constraint at beta=1/2 only C1(grid)=0.02250 worst violation=1.8e-06 it=59  2s
m=20 d=0.70 constraint at beta=1/2 only C1(grid)=0.01070 worst violation=5.4e-01 it=59  3s   # NOT CONVERGED (not quoted)
m=20 d=0.80 constraint at beta=1/2 only C1(grid)=0.01257 worst violation=7.6e-02 it=59  4s   # NOT CONVERGED (not quoted)
m=20 d=0.90 constraint at beta=1/2 only C1(grid)=0.01330 worst violation=1.3e-02 it=59  4s   # NOT CONVERGED (not quoted)
```

S3. Input data of the certificates

The following files hold the coefficients as decimal strings that round-trip to the binary floating-point numbers produced by the optimiser (the certifiers convert them to exact rationals), the disc data before rounding to rationals (the certificate rounds c_k, R_k, v_k to denominators $\leq 10^4$ and then replaces R_k by $\min(R_k, c_k - \frac{1}{2})$, as in `certify_hybrid.py`), and the parameters of the search. The fields `J_grid` and `C_grid_objective` are the floating-point values of the search programme and are not used by the certificates, which recompute J_k and C_{H} exactly; they are kept so that the file is the unaltered output of the search (its digest is part of the provenance lines of the certificate). The rational disc data actually certified are those of Table 4 of the article, obtained from the decimal values below by the two lines of `certify_hybrid.py` quoted in Appendix S1.2.

S3.1. **`hyb_m10_P.json` (hybrid).**

```
{
 "m": 10,
 "d": 0.9,
 "prof": "P",
 "eta": 0.001,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["R", 6],
  ["R", 7],
  ["R", 8],
  ["R", 9],
  ["R", 10],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5],
  ["I", 6],
  ["I", 7],
  ["I", 8],
  ["I", 9],
  ["I", 10]
 ],
 "c": [0.41206500728461565, 0.37015428795149224, 0.2844015811499438, 0.15861857578668956,
  0.06954223006530842, -0.06157764993877496, -0.040570135781007474, 0.07526718292596676,
  -0.03235073058726206, 0.005632605237216712, 0.7334607489350747, -0.13613005947680004,
  0.15136050241561017, -0.11692974368156732, -0.03656894918659249, 0.10601427043647423,
  -0.06227163099672864, 0.016690927091753422, -0.0017941465528435766],
 "discs": [
  [1.03, 0.18000000000000002, -0.5],
  [1.03, 0.18000000000000002, 0.5],
  [1.03, 0.23000000000000004, -0.65],
  [1.03, 0.23000000000000004, 0.65],
  [1.03, 0.28, -0.65],
```

```
[1.03, 0.28, 0.65],
[1.03, 0.33, -0.65],
[1.03, 0.33, 0.65],
[1.03, 0.38, -0.65],
[1.03, 0.38, 0.65],
[1.03, 0.43000000000000005, 1.0],
[1.03, 0.48000000000000004, -1.0],
[1.03, 0.48000000000000004, 1.0],
[1.03, 0.53, -1.6],
[1.03, 0.53, -1.3],
[1.03, 0.53, -1.0],
[1.03, 0.53, 1.0],
[1.03, 0.53, 1.3],
[1.06, 0.16000000000000006, -0.5],
[1.06, 0.16000000000000006, 0.5],
[1.06, 0.21000000000000005, -0.5],
[1.06, 0.21000000000000005, 0.5],
[1.06, 0.26000000000000006, -0.5],
[1.06, 0.26000000000000006, 0.5],
[1.06, 0.26000000000000006, 0.65],
[1.06, 0.41000000000000003, -0.8],
[1.06, 0.41000000000000003, -0.65],
[1.06, 0.41000000000000003, 0.65],
[1.06, 0.41000000000000003, 0.8],
[1.06, 0.4600000000000001, 0.8],
[1.06, 0.56, -1.6],
[1.06, 0.56, -1.3],
[1.06, 0.56, 1.3],
[1.1, 0.15000000000000008, -0.35],
[1.1, 0.2500000000000001, -0.5],
[1.1, 0.2500000000000001, 0.5],
[1.1, 0.4000000000000001, 0.8],
[1.1, 0.45000000000000007, -0.8],
[1.1, 0.45000000000000007, 0.8],
[1.1, 0.5000000000000001, -0.8],
[1.1, 0.5000000000000001, 0.8],
[1.1, 0.6000000000000001, -1.0],
[1.1, 0.6000000000000001, 1.0],
[1.1, 0.6000000000000001, 1.3],
[1.0986122886681098, 0.1986122886681098, -0.5],
[1.0986122886681098, 0.1986122886681098, 0.5],
[1.0986122886681098, 0.24861228866810978, -0.5],
[1.0986122886681098, 0.24861228866810978, 0.5],
[1.0986122886681098, 0.2986122886681098, -0.5],
[1.0986122886681098, 0.2986122886681098, 0.5],
[1.0986122886681098, 0.44861228866810976, -0.8],
[1.0986122886681098, 0.44861228866810976, 0.8],
[1.0986122886681098, 0.4986122886681098, -0.8],
[1.0986122886681098, 0.4986122886681098, 0.8],
[1.0986122886681098, 0.5986122886681098, -1.3],
[1.15, 0.1999999999999999, -0.5],
[1.15, 0.1999999999999999, -0.35],
[1.15, 0.1999999999999999, 0.35],
[1.15, 0.4499999999999999, -0.8],
[1.15, 0.4999999999999999, -0.8],
[1.15, 0.5499999999999999, -0.8],
[1.15, 0.5499999999999999, 0.8],
[1.15, 0.6499999999999999, -1.3],
[1.15, 0.6499999999999999, -1.0],
[1.15, 0.6499999999999999, 1.0],
[1.15, 0.6499999999999999, 1.3],
[1.2, 0.24999999999999994, -0.5],
[1.2, 0.24999999999999994, -0.35],
[1.2, 0.24999999999999994, 0.35],
[1.2, 0.24999999999999994, 0.5],
[1.2, 0.7, -1.3],
[1.2, 0.7, -1.0],
[1.2, 0.7, 1.0],
[1.2, 0.7, 1.3],
[1.25, 0.3, -0.35],
[1.25, 0.3, 0.35],
[1.25, 0.75, -1.3],
[1.25, 0.75, -1.0],
[1.25, 0.75, 1.0],
[1.25, 0.75, 1.3],
[1.3, 0.8, -1.0],
[1.3, 0.8, 1.0],
[1.4, 0.4499999999999999, -0.35],
[1.4, 0.4499999999999999, 0.35],
[1.4, 0.8999999999999999, -1.3],
[1.4, 0.8999999999999999, 1.3],
```

```
  [1.5, 0.55, -0.35],
  [1.5, 0.55, 0.35],
  [1.6094379124341003, 0.6594379124341003, -0.35],
  [1.6094379124341003, 0.6594379124341003, 0.35]
 ],
 "kappa": [0.04645216860607228, 0.04300701194678176, 0.047063043236809085,
  0.14669081272175175, 0.025979954843667673, 0.10379166553442853, 0.0238180047300216,
  0.1041736978433328, 0.014539531796755864, 0.0648721619755471, 0.01585298753435496,
  0.0033617963250462696, 0.006488928258029474, 5.4144875650491944e-05,
  0.0003018716070398441, 0.0009151843658643538, 0.004457480242890296,
  0.0014660000151806882, 0.047525841386079126, 0.15907294401238437, 0.025389681493291277,
  0.2223325253406065, 0.08631620266291855, 0.1538221037360442, 0.011498324806702449,
  0.010728457418492028, 0.024094460474547018, 0.08996010049122434, 0.018772230071276146,
  0.013598324363960737, 9.669233392673498e-05, 4.8027806973018017e-05,
  0.0001824982987809 1896, 0.05671657488344152, 0.051472319257400555, 0.18833803873166002,
  0.07545736055574893, 0.0016274867554092252, 0.009838802010945192, 0.0012561800451027275,
  0.005913769696587884, 0.0006183338065604295, 0.002725978105029675,
  0.00021124645907630063, 0.05954976707208953, 0.25468461930865344, 0.016700547594809507,
  0.07440766182674931, 0.01266957077706443, 0.18918149312114735, 0.0014777755579662957,
  0.006728141614374093, 0.0014793988972288392, 0.0054601805478273185,
  5.5623684826215334e-05, 0.05424339889920384, 0.03475948981696701, 0.21438371806294945,
  0.00631637069925039, 0.0033326420626991556, 0.006954420904420063, 0.028592802039103745,
  5.25882403645087e-05, 0.0004219252908076343, 0.0018326737860931267,
  0.00020626507835385952, 0.03706538770583855, 0.030599996800513854, 0.06841653547695875,
  0.332325423847939, 4.57943912298433e-05, 0.0003653681965739939, 0.0015721607201088414,
  0.0001785005555963715, 0.05227534137638917, 0.2040492822717576, 6.871033611742962e-05,
  0.0003184030759719035, 0.001362286750857658, 0.0002746990132641945,
  0.0008212433177690405, 0.003961667358838006, 0.3861919018728296, 0.8551654872594248,
  0.00014937245314941035, 0.0014320991096890576, 0.09799147070095049, 0.33882316551798763,
  0.16714072019417897, 0.5061513200725676],
 "J_grid": [0.006913876985130073, 0.006913876985130073, 0.010786963484310835,
  0.010786963484310835, 0.015048973174851672, 0.015048973174851672, 0.019571517019666976,
  0.019571517019666976, 0.024737702602379327, 0.024737702602379327, 0.030311093588762966,
  0.03614382786661558, 0.03614382786661558, 0.042238079000552084, 0.042238079000552084,
  0.042238079000552084, 0.042238079000552084, 0.042238079000552084, 0.0032593310610211064,
  0.0032593310610211064, 0.006361647634553729, 0.006361647634553729, 0.010091729993481463,
  0.010091729993481463, 0.02372706874438562, 0.02372706874438562,
  0.02372706874438562, 0.02372706874438562, 0.029208025469220297, 0.04097579484229039,
  0.04097579484229039, 0.04097579484229039, 0.0007855777381792925, 0.00579739181011531,
  0.00579739181011531, 0.01762364909587594, 0.022555747215725654, 0.022555747215725654,
  0.027910450281615512, 0.027910450281615512, 0.039458155882717894, 0.039458155882717894,
  0.039458155882717894, 0.0029087213249446332, 0.0029087213249446332, 0.005814504320743654,
  0.005814504320743654, 0.009368782031364846, 0.009368782031364846, 0.02259349298288061,
  0.02259349298288061, 0.027952579183055815, 0.027952579183055815, 0.03950798254330579,
  0.0006778485850694062, 0.0006778485850694062, 0.0006778485850694062, 0.01654368154193419,
  0.021311247069162823, 0.026509942596292888, 0.026509942596292888, 0.037780805766164105,
  0.037780805766164105, 0.037780805766164105, 0.037780805766164105, 0.0006049812018696226,
  0.0006049812018696226, 0.0006049812018696226, 0.0006049812018696226, 0.03630213022245224,
  0.03630213022245224, 0.03630213022245224, 0.03630213022245224, 0.0005514853690123469,
  0.0005514853690123469, 0.034985510865448044, 0.034985510865448044, 0.034985510865448044,
  0.034985510865448044, 0.033803191348382326, 0.033803191348382326, 0.00044923232543557626,
  0.00044923232543557626, 0.031760036845278866, 0.031760036845278866, 0.00040603160664041,
  0.00040603160664041, 0.00037055380885643406, 0.00037055380885643406],
 "C_grid_objective": 0.07485961386282106
}
```


S3.2. **`hyb_m10_P_discs_rational.json` (derived).** The rational disc data (c_k, R_k, v_k) and the exact weights κ_k used by the certificate, as pairs (numerator, denominator), obtained from `hyb_m10_P.json` by the conversions of `certify_hybrid.py`; the file is generated deterministically by `gen_appendix.py` (keys sorted, one value per line), is not an input of any certificate, and is included in the deposited archive; its SHA-256 is `bea13456c6d5f11359a86d8ee6b1b0a684b12e13a46bc4d12db11933e28b323a`. Its first lines are


```
{
 "d": [
  9,
  10
 ],
 "discs": [
  {
   "R": [
    9,
    50
   ],
   "c": [
...
```

S3.3. `line_L5/sol.json` **(line ℓ_5).**


```
{
 "d": 1.0,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [1.2676737932696245, 0.7009294514561635, -0.4946330486621655, 0.09751555052498845,
  -0.0005893962931625271, -0.3784073335097401, 0.13473062893139318, -0.01708382778797021,
  0.0001003433684765934],
 "C1": 0.20175655042691418,
 "Cz_est": 0.8391011111542401,
 "z_est": 0.665362725928392,
 "A3_est": 1.504463837082632,
 "Lstar": 5.0,
 "eta": 0.0002,
 "discs": [],
 "kappa": [],
 "m": 5,
 "val": 2.5132465892172027,
 "PPMAX": 0
}
```


S3.4. `line_L20/sol.json` **(line ℓ_{20}).**


```
{
 "d": 1.2,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [0.6799784749489147, 0.8041106391650762, 0.7774270907421565, -0.008448613496901295,
  2.955972158446658e-05, 1.1843502097112268, -0.576537489831307, 0.00610293149500534,
  -2.1076986306361527e-05],
 "C1": 0.1082219354842083,
 "Cz_est": 1.6797641910534995,
 "z_est": 0.7020688851693405,
 "A3_est": 2.38183307622284,
 "Lstar": 20.0,
 "eta": 0.0002,
 "discs": [],
 "kappa": [],
 "m": 5,
 "val": 4.546271785907006,
 "PPMAX": 0
}
```


S3.5. `line_L60/sol.json` **(line ℓ_{60}).**


```
{
 "d": 1.0,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [0.5951532035147798, 0.5911312495450366, 0.3889216428262093, -0.004325167636924481,
  1.5378902475155497e-05, 0.8079466147098463, -0.3400480079099986, 0.003649167179891613,
  -1.2750467413103112e-05],
 "C1": 0.094721574236354,
 "Cz_est": 2.344884047970574,
```

```
 "z_est": 0.7289397780667036,
 "A3_est": 3.0738238260372777,
 "Lstar": 60.0,
 "eta": 0.0002,
 "discs": [],
 "kappa": [],
 "m": 5,
 "val": 8.757118280218517,
 "PPMAX": 0
}
```


S3.6. `line_L150/sol.json` **(line ℓ_{150}).**


```
{
 "d": 0.85,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5]
 ],
 "c": [0.5873561110952275, 0.4955273589230774, 0.17191023951649326, -0.04515429156322006,
  0.0002717467554019091 7, 0.5292986070563637, -0.26974254048270235, 0.03950670966367516,
  -0.00023459073769482711],
 "C1": 0.09348062843603788,
 "Cz_est": 2.5047342757708364,
 "z_est": 0.7303153300893256,
 "A3_est": 3.235049605860162,
 "Lstar": 150.0,
 "eta": 0.0002,
 "discs": [],
 "kappa": [],
 "m": 5,
 "val": 17.257143871265846,
 "PPMAX": 0
}
```


S3.7. `line_C400/sol.json` **(line ℓ_{400}).**


```
{
 "d": 0.85,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["R", 6],
  ["R", 7],
  ["R", 8],
  ["R", 9],
  ["R", 10],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5],
  ["I", 6],
  ["I", 7],
  ["I", 8],
  ["I", 9],
  ["I", 10]
 ],
 "c": [0.5276997168610267, 0.44844462233410926, 0.20734915440049123, -0.05170881362706953,
  0.04277439204584691, -0.0021686787194867547, 5.225366348519004e-05,
  -7.194711787780742e-07, 5.548232880956561e-09, -1.890562024872484e-11, 0.57950165380938,
  -0.24304154954646912, 0.09598460587810165, -0.022316290439150017, 0.0010166745274645633,
  -2.326790970420105e-05, 3.096452357377663e-07, -2.3274894920069496e-09,
  7.769753796000765e-12],
 "C1": 0.08398601840662599,
 "Cz_est": 4.723835786462193,
 "z_est": 0.7375635792618767,
 "A3_est": 5.46139936572407,
 "Lstar": 400.0,
 "eta": 0.0002,
 "discs": [],
 "kappa": [],
```

```
 "m": 10,
 "val": 39.05580672837446,
 "PPMAX": 0
}
```


S3.8. `line_D6000/sol.json` **(line ℓ_{6000}).**


```
{
 "d": 0.85,
 "B": [
  ["R", 1],
  ["R", 2],
  ["R", 3],
  ["R", 4],
  ["R", 5],
  ["R", 6],
  ["R", 7],
  ["R", 8],
  ["R", 9],
  ["R", 10],
  ["I", 2],
  ["I", 3],
  ["I", 4],
  ["I", 5],
  ["I", 6],
  ["I", 7],
  ["I", 8],
  ["I", 9],
  ["I", 10]
 ],
 "c": [0.5266199652605132, 0.44752559745950127, 0.21763335933091524, -0.06632610210813339,
  0.049224751946469816, -0.002454597973796714, 5.8415182530844107e-05,
  -7.96150671490144e-07, 6.08645994127779e-09, -2.0583832928014207e-11, 0.5834246015810649,
  -0.23667212703027896, 0.09567840750457196, -0.01858966229544025, 0.0008313848476117577,
  -1.89384002058713e-05, 2.517567194510178e-07, -1.8927048877024136e-09,
  6.322548809318692e-12],
 "C1": 0.08381417060209288,
 "Cz_est": 4.566001466369842,
 "z_est": 0.7357358598268879,
 "A3_est": 5.30173732619673,
 "Lstar": 6000.0,
 "eta": 0.0002,
 "discs": [],
 "kappa": [],
 "m": 10,
 "val": 508.18676093875405,
 "PPMAX": 300
}
```


References

[1] The FLINT team, FLINT: Fast Library for Number Theory, version 3.6.0, `https://flintlib.org`.

[2] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, *IEEE Trans. Comput.* **66** (2017) 1281–1292, doi:10.1109/TC.2017.2690633.

Universidad Nacional de Educación a Distancia, Madrid, Spain
Email address: `vcasana1@alumno.uned.es`